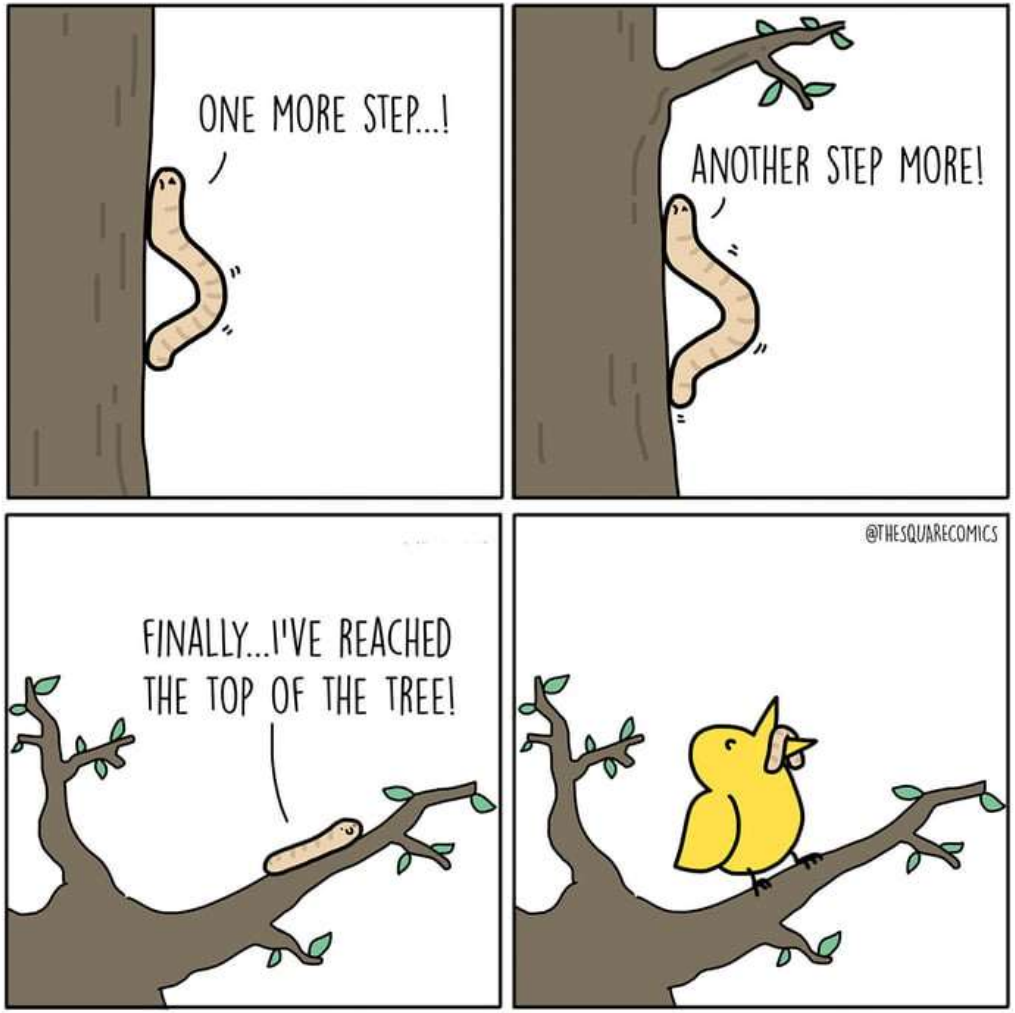
Tehnologii Web

servicii Web (II)

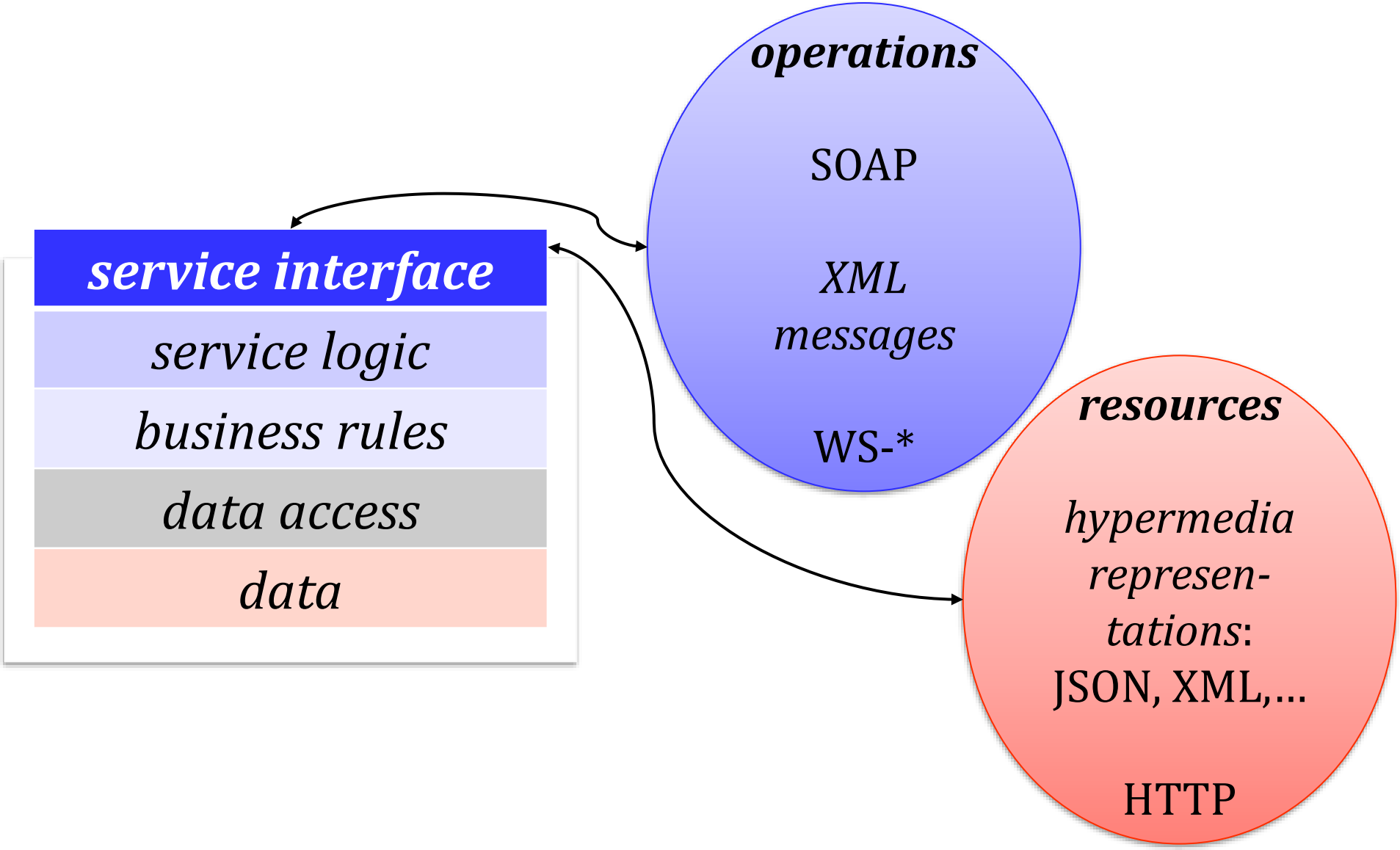
dezvoltarea
de aplicații Web
cu **REST**



Dr. Sabin Corneliu Buraga – **profs.info.uaic.ro/~busaco/**

„Concizia este sora talentului.”

Anton Cehov



serviciile Web pot fi dezvoltate via **SOAP** și/sau **REST**

Stefan Tilkov, *REST: Not an Intro* (2013)

speakerdeck.com/stilkov/rest-not-an-intro-1

Există o modalitate de creare/invocare a serviciilor Web fără a recurge la SOAP?

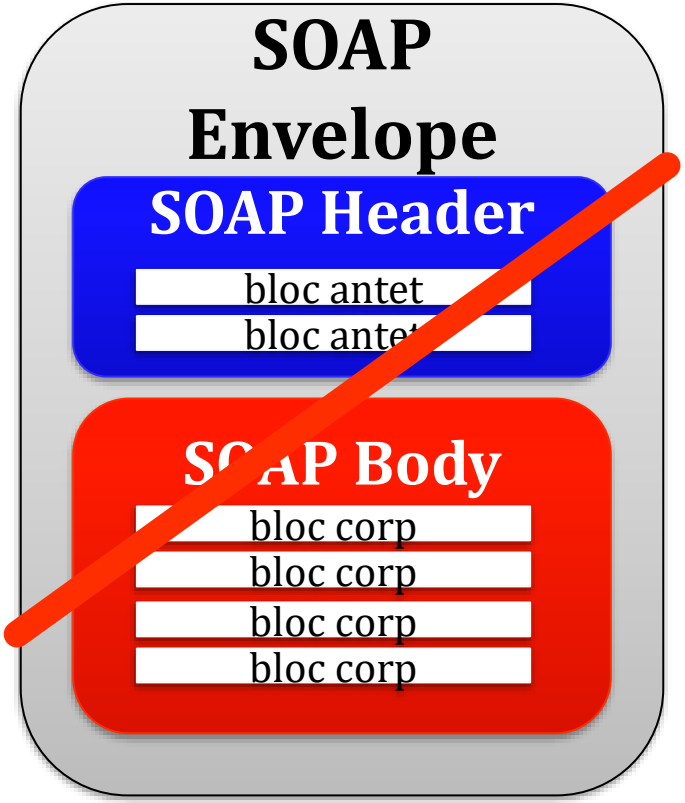
rest: representational state transfer

Stil arhitectural de dezvoltare a aplicațiilor Web axat asupra reprezentării datelor

Roy Fielding
teză de doctorat, 2000 (*University of California*, Irvine)
www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

rest

Viziune complementară de implementare și utilizare a serviciilor Web – fără SOAP



rest

Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

rest

Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

resursă Web

utilizator având cont în cadrul unui sistem,
mesaj al unei persoane, fotografie, flux de știri,
componentă software, set de date (*dataset*), model 3D,...

rest

Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

reprezentare pe baza unui format de date

textual sau binar

exemple tipice – formate deschise:
HTML, JSON, CSV, PNG, SVG, PDF etc.

rest

Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

reprezentare pe baza unui format de date

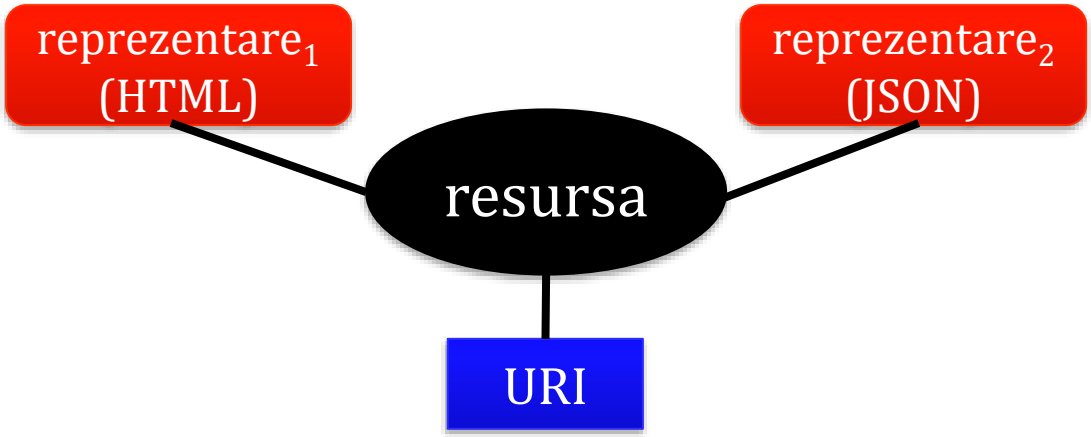
formatul reprezentării e desemnat de tipuri MIME
text/html, **text/xml**, **text/csv**, **application/json**, **image/png**

detalii în N.Freed *et al.*, *Media Types*, 19 aprilie 2021
www.iana.org/assignments/media-types/media-types.xhtml

rest

Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

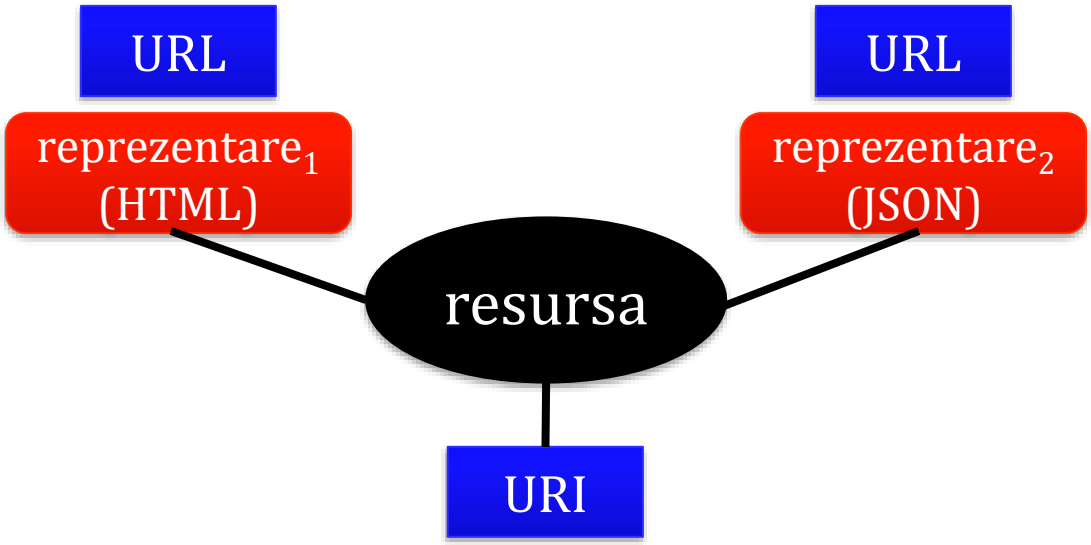
reprezentările aceleași resurse
– desemnate de un URI unic – pot fi multiple



rest

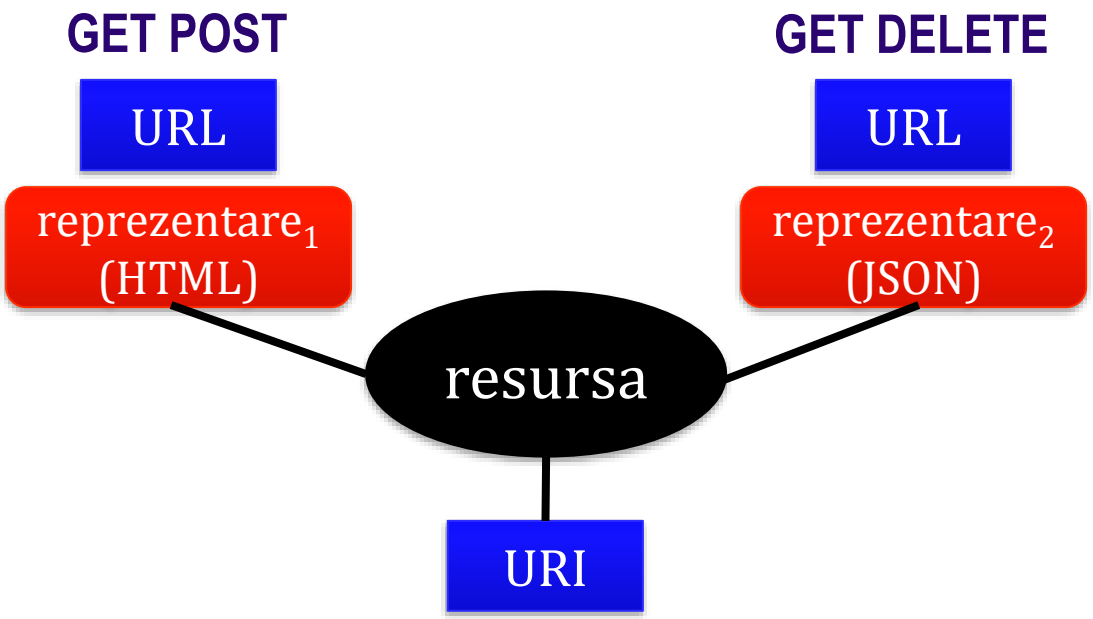
Rezultatul unei procesări conduce la obținerea unei **reprezentări** a unei resurse

fiecare reprezentare a unei resurse are asociat un URL



rest

Clienții (*e.g.*, navigatoare Web, roboți, *player*-e etc.) interacționează cu reprezentările resurselor via **verbe** „accesează”: **GET**, „modifică”: **POST**, „șterge”: **DELETE**,...



rest

Verbele (acțiunile) sunt stipulate de un protocol

de obicei, HTTP (*HyperText Transfer Protocol*)

de (re)parcurs
cursul al doilea

rest

GET

accesează (preia) o reprezentare a unei resurse

nu conduce la modificarea stării serverului – *safe*

idempotentă – cereri identice vor conduce la oferirea aceluiași răspuns (aceeași reprezentare)

rest

HEAD

similară cu GET, dar furnizează doar meta-date
(nu oferă reprezentarea propriu-zisă)

e.g., ultima actualizare, lungimea conținutului,…

rest

PUT

înlocuiește (actualizează) o reprezentare de resursă sau eventual creează o resursă la nivel de server Web (al cărei URI e deja cunoscut)

uzual, returnează URI-ul resursei

nu e considerată *safe*, dar este idempotentă

rest

PATCH

permite actualizarea parțială a unei reprezentări
a unei resurse (PUT nu oferă o asemenea facilitate)

nu este *safe* și nici idempotentă

rest

POST

creează o resursă (uzual, subordonată altei resurse)

opțional, pot fi realizate procesări suplimentare

nu este nici *safe*, nici idempotentă

clientul nu cunoaște *a-priori*
care va fi URI-ul resursei ce va fi create

rest

DELETE

șterge (elimină) o resursă desemnată de un URI

este idempotentă

rest

OPTIONS

permite clientului să determine diverse cerințe privitoare
la o resursă (*e.g.*, dacă o resursă poate fi ștearsă)
sau facilitățile expuse de un server
(de exemplu, suportul oferit de un *proxy*)

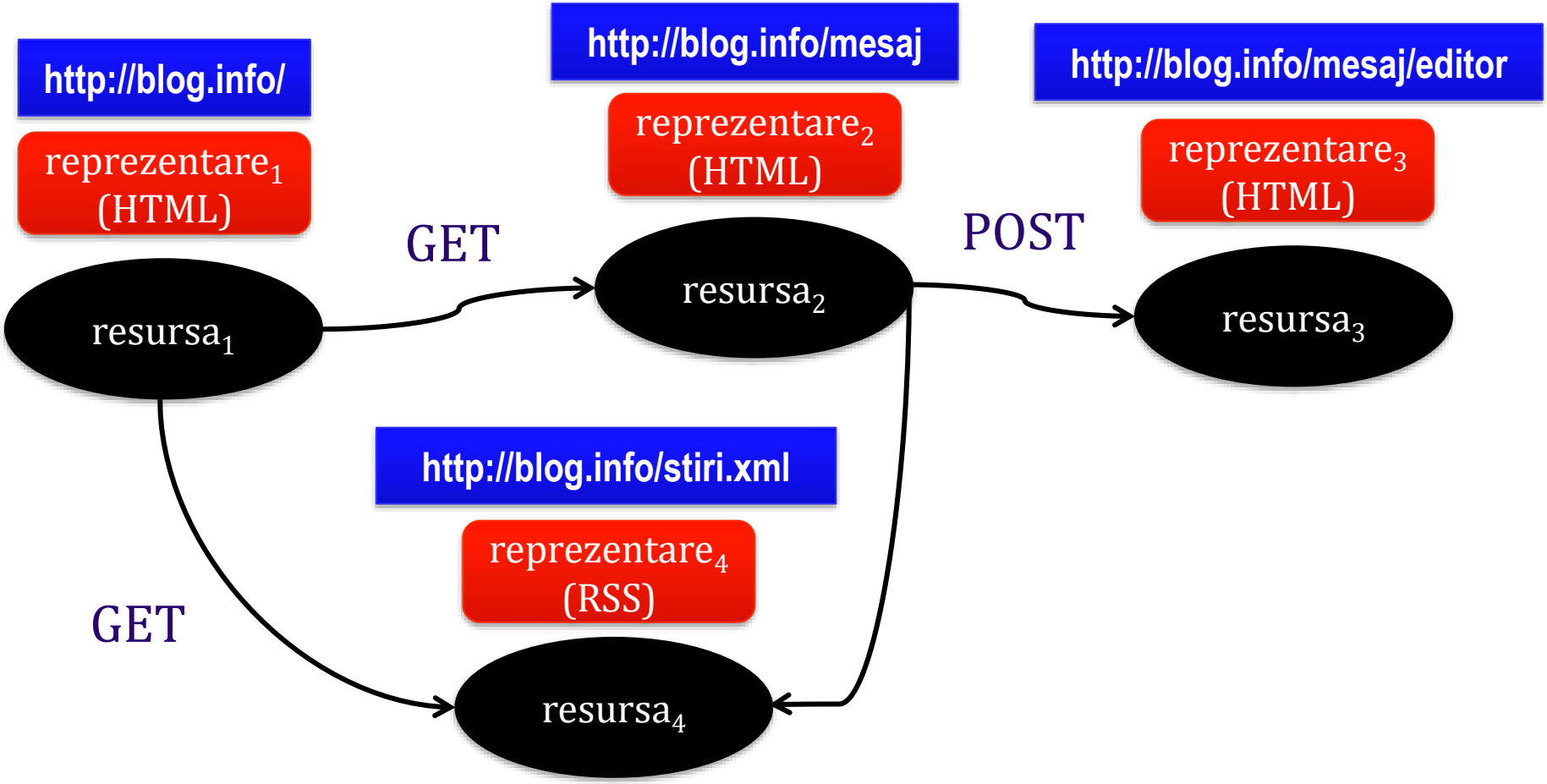
avansat

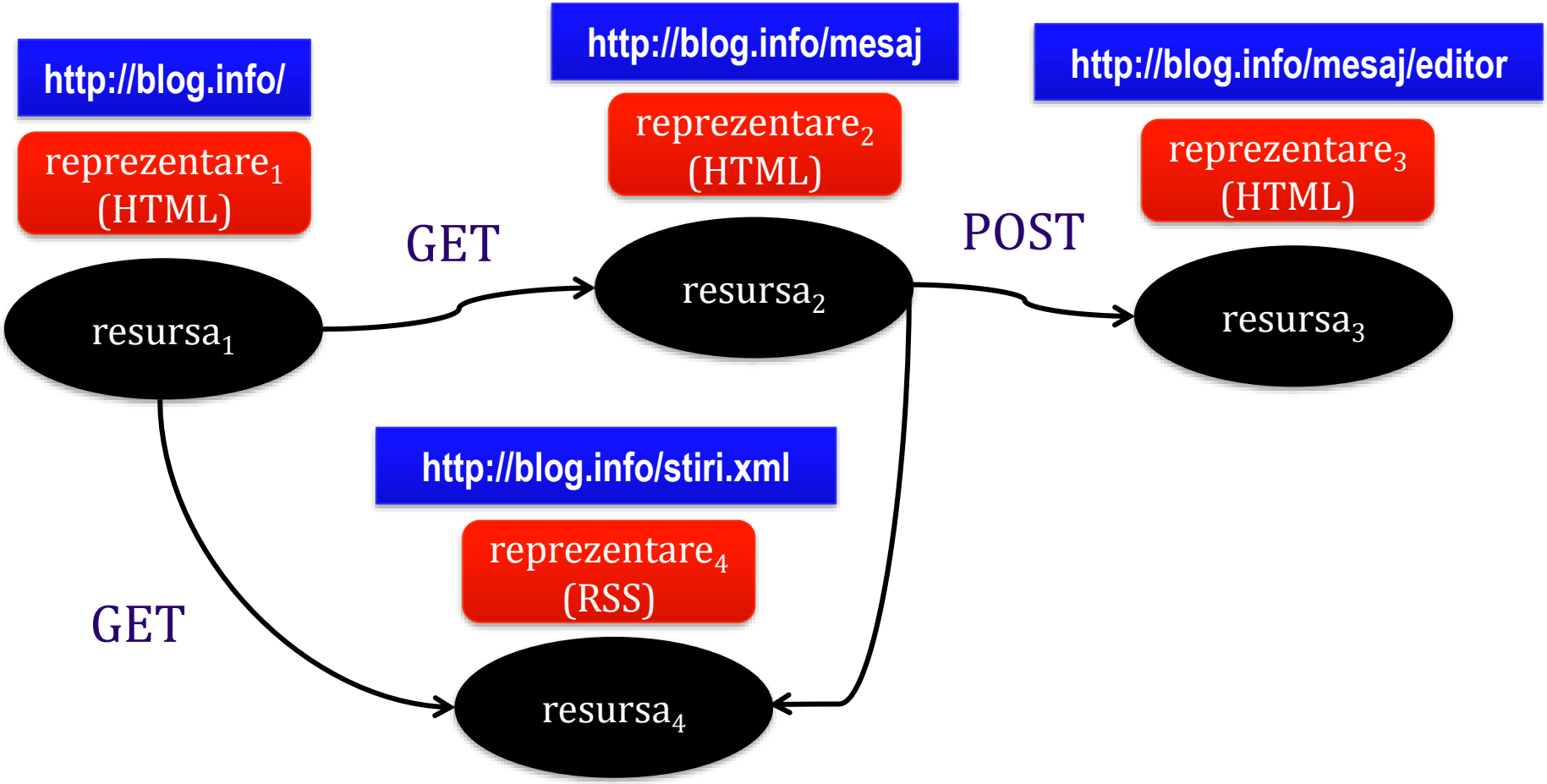
Metoda HTTP	*Idempotent*	*Safe*
GET	✓	✓
POST	X	X
PUT	✓	X
PATCH	X	X
OPTIONS	✓	✓
HEAD	✓	✓
DELETE	✓	X

de studiat și articolul Tamas Piros,
RESTful API Design – POST vs PUT vs PATCH, 2018
tpiros.dev/blog/restful-api-design-post-vs-put-vs-patch/

rest

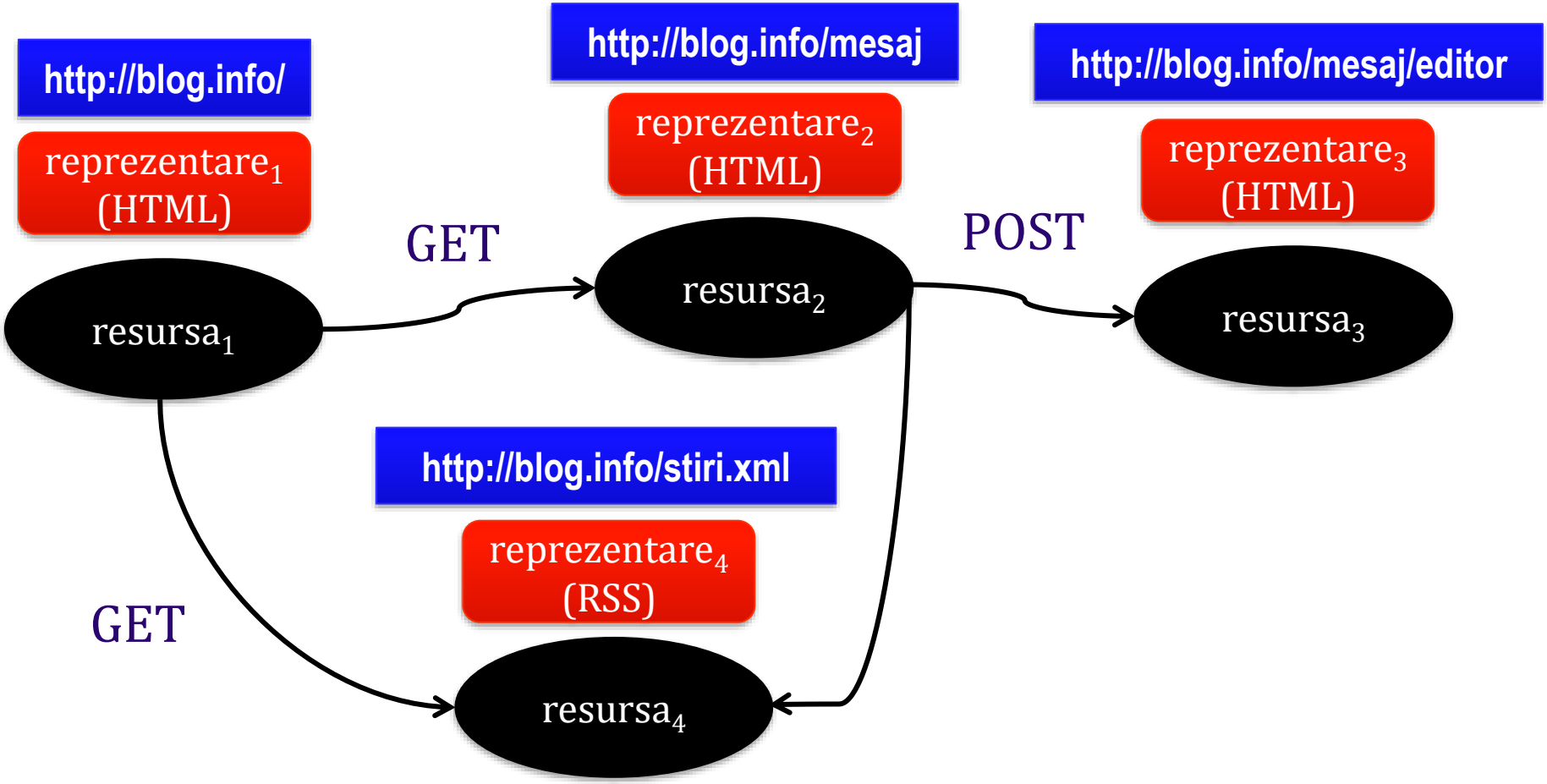
Orice accesare a unei reprezentări
plasează aplicația – ori clientul Web – într-o **stare**
ce va fi schimbată în urma unui **transfer** de date
(accesarea altei reprezentări)


http://blog.info/
reprezentare1 (HTML)
resursa1
GET
http://blog.info/mesaj
reprezentare2 (HTML)
resursa2
POST
http://blog.info/mesaj/editor
reprezentare3 (HTML)
resursa3
http://blog.info/stiri.xml
reprezentare4 (RSS)
resursa4
GET



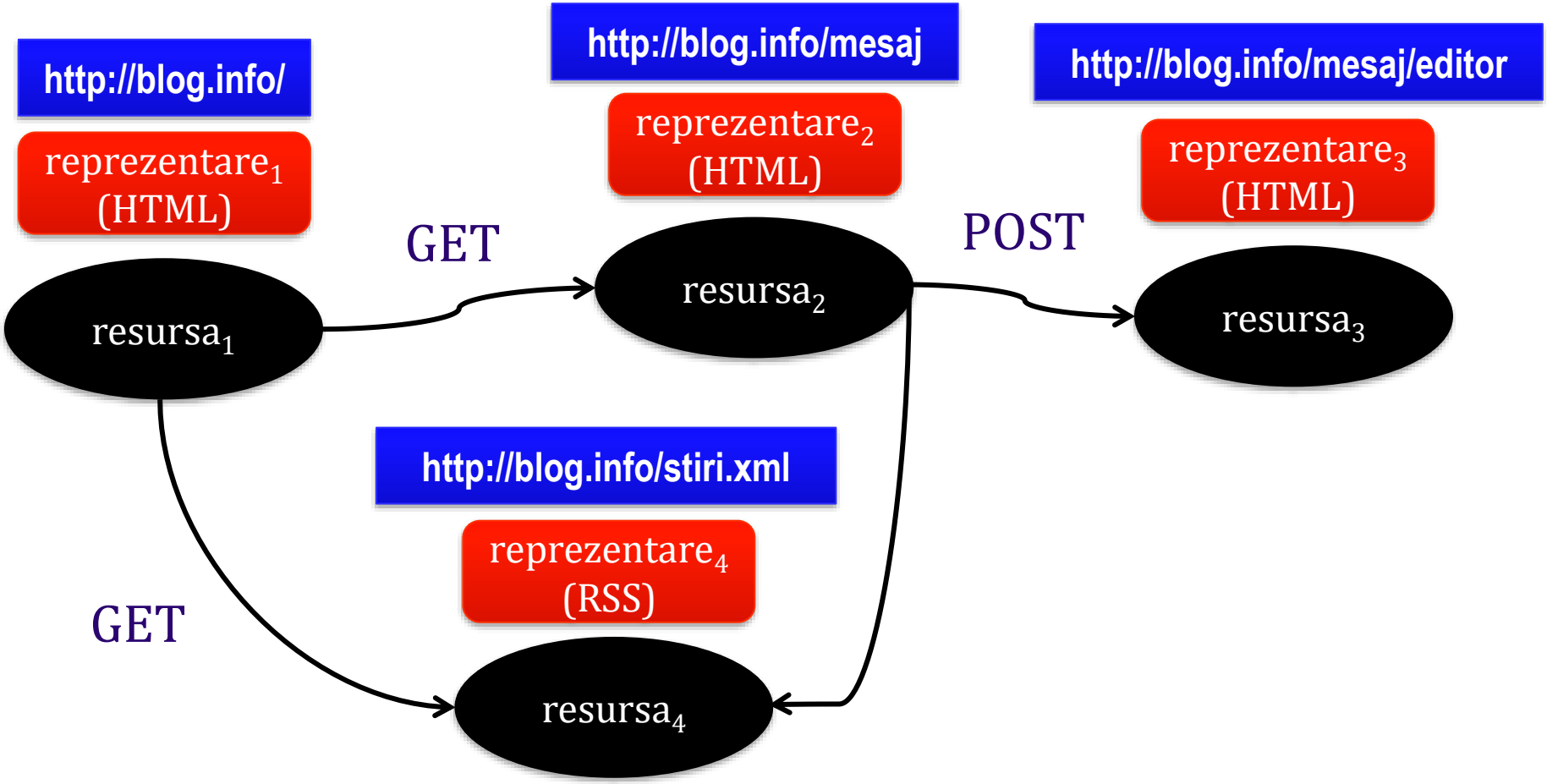
aplicație Web ≡ automat finit nedeterminist

tranzițiile dintre stări sunt dictate de transferuri de date pe baza metodelor protocolului – uzual, HTTP



pe baza hipermediei – graful de legături între resurse –,
dintr-o anumită stare (reprezentare)
se pot realiza tranziții la alte stări

utilizatorul/programul accesează altă resursă via un URI



HATEOAS (*Hypermedia As The Engine Of Application State*)

B. Doerrfeld, *Designing a True REST State Machine* (2017)

nordicapis.com/designing-a-true-rest-state-machine/

rest

Transferul se realizează prin protocolul HTTP

Reprezentarea este modelată conform unui format
– *e.g.*, JSON sau XML – și
indicată prin tipuri MIME (*media types*)

Adresabilitatea se rezolvă via URI

rest

Aplicațiile care invocă funcționalități (servicii) consumă reprezentări de resurse – în stilul *pull*

rest

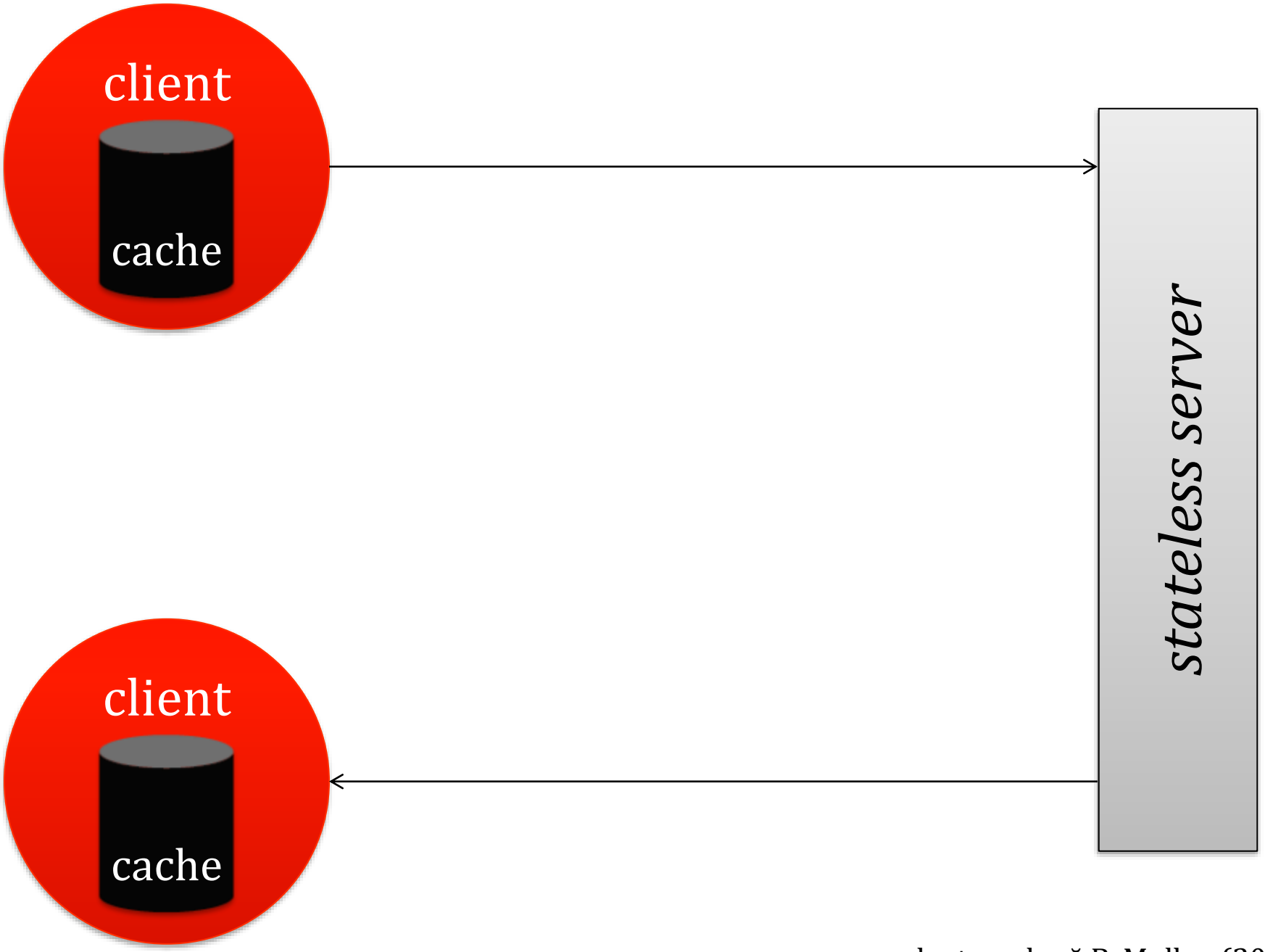
Fiecare cerere este considerată independentă, fără a se lua în calcul contextul

stateless server

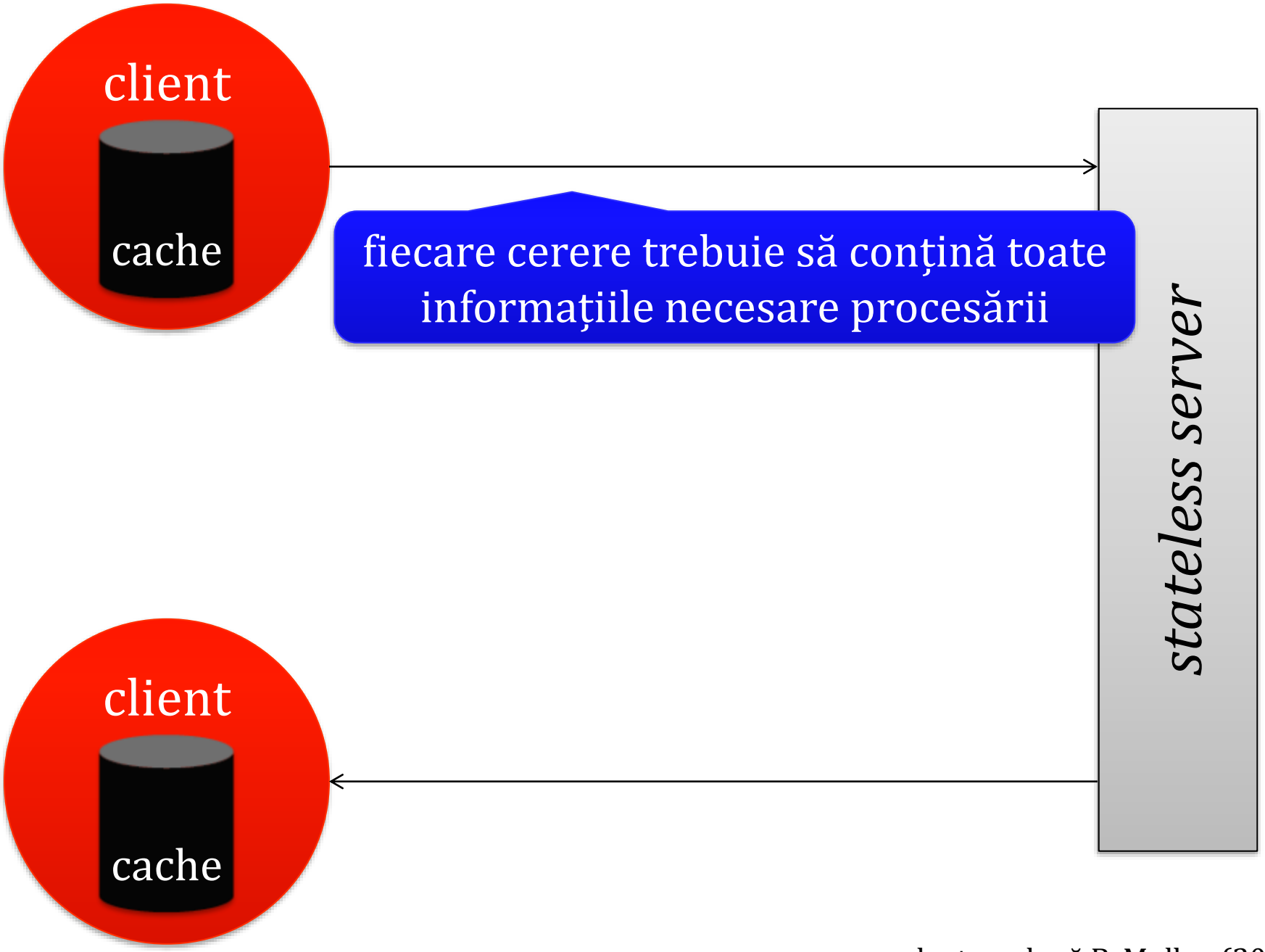
rest

Reprezentările de resurse pot fi stocate temporar

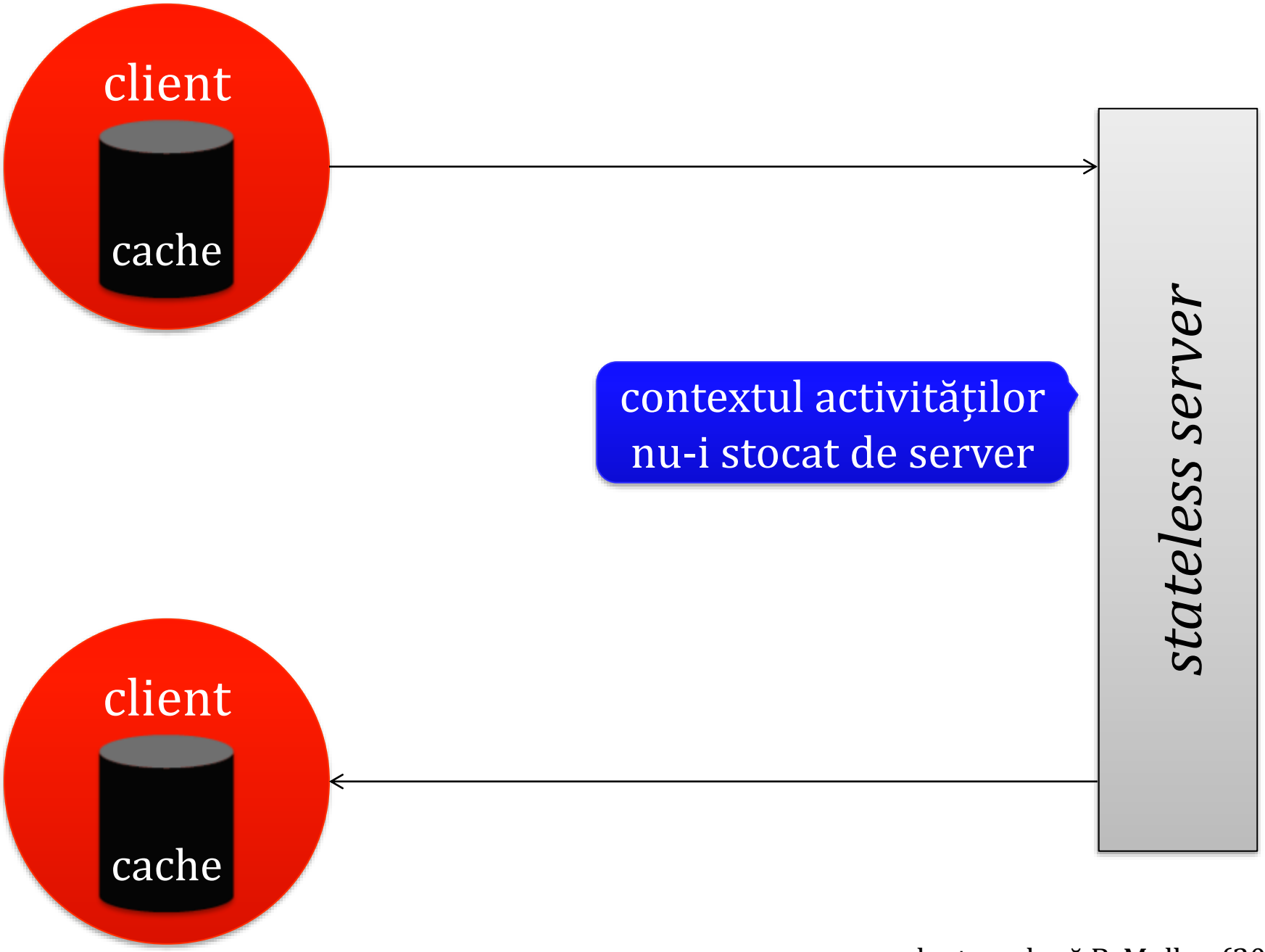
caching



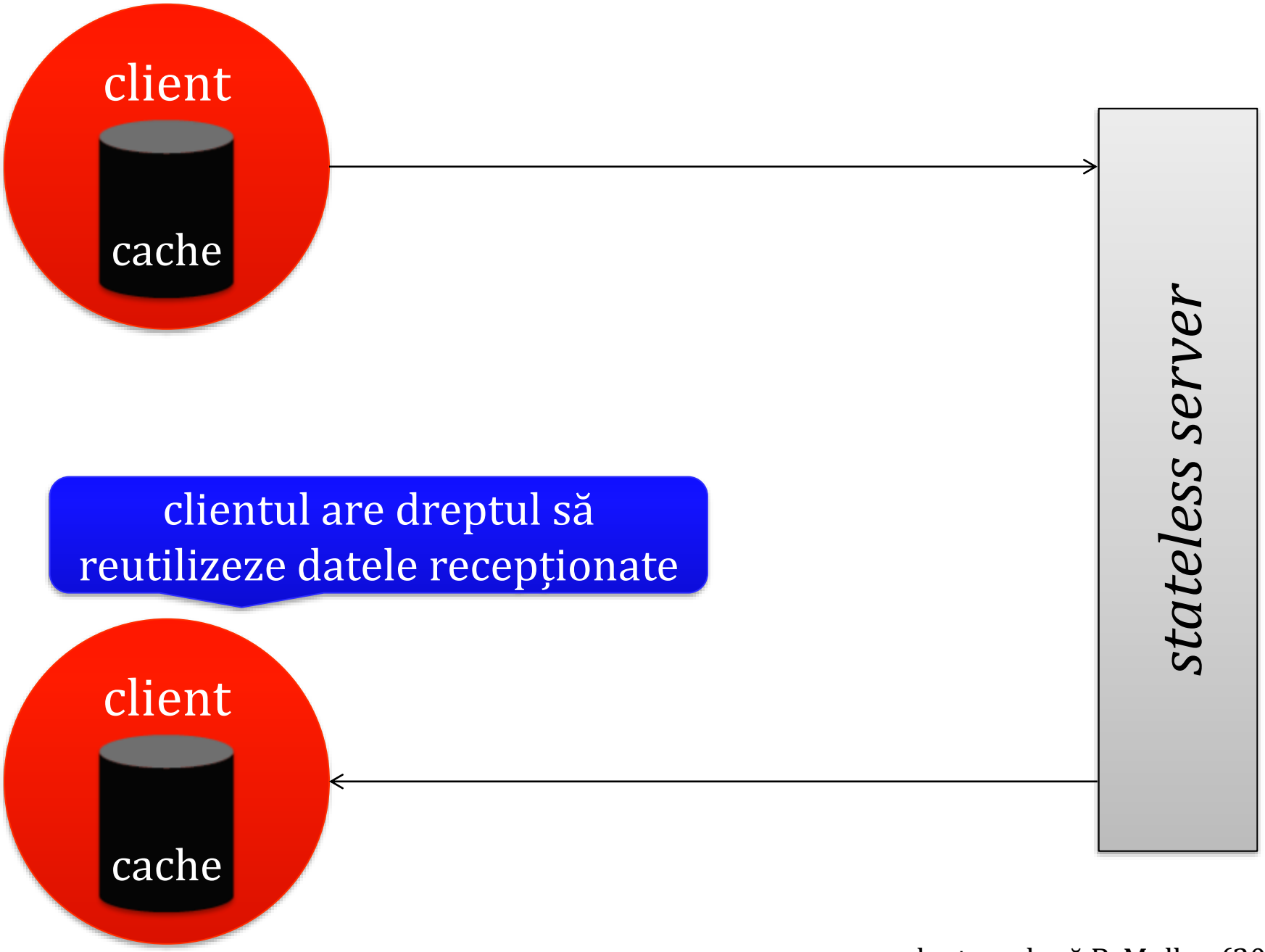
adaptare după B. Mulloy (2012)



adaptare după B. Mulloy (2012)



adaptare după B. Mulloy (2012)

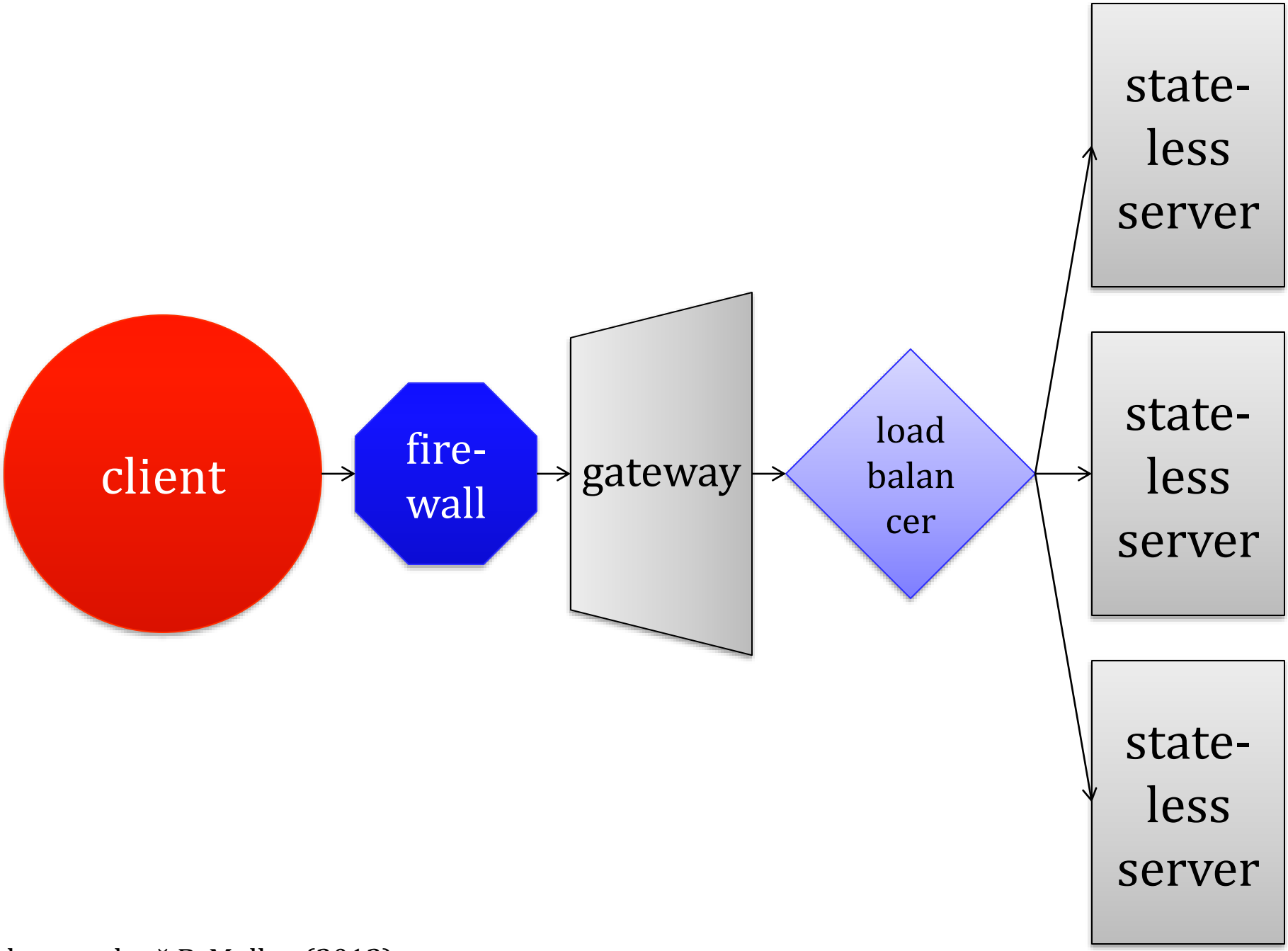


adaptare după B. Mulloy (2012)

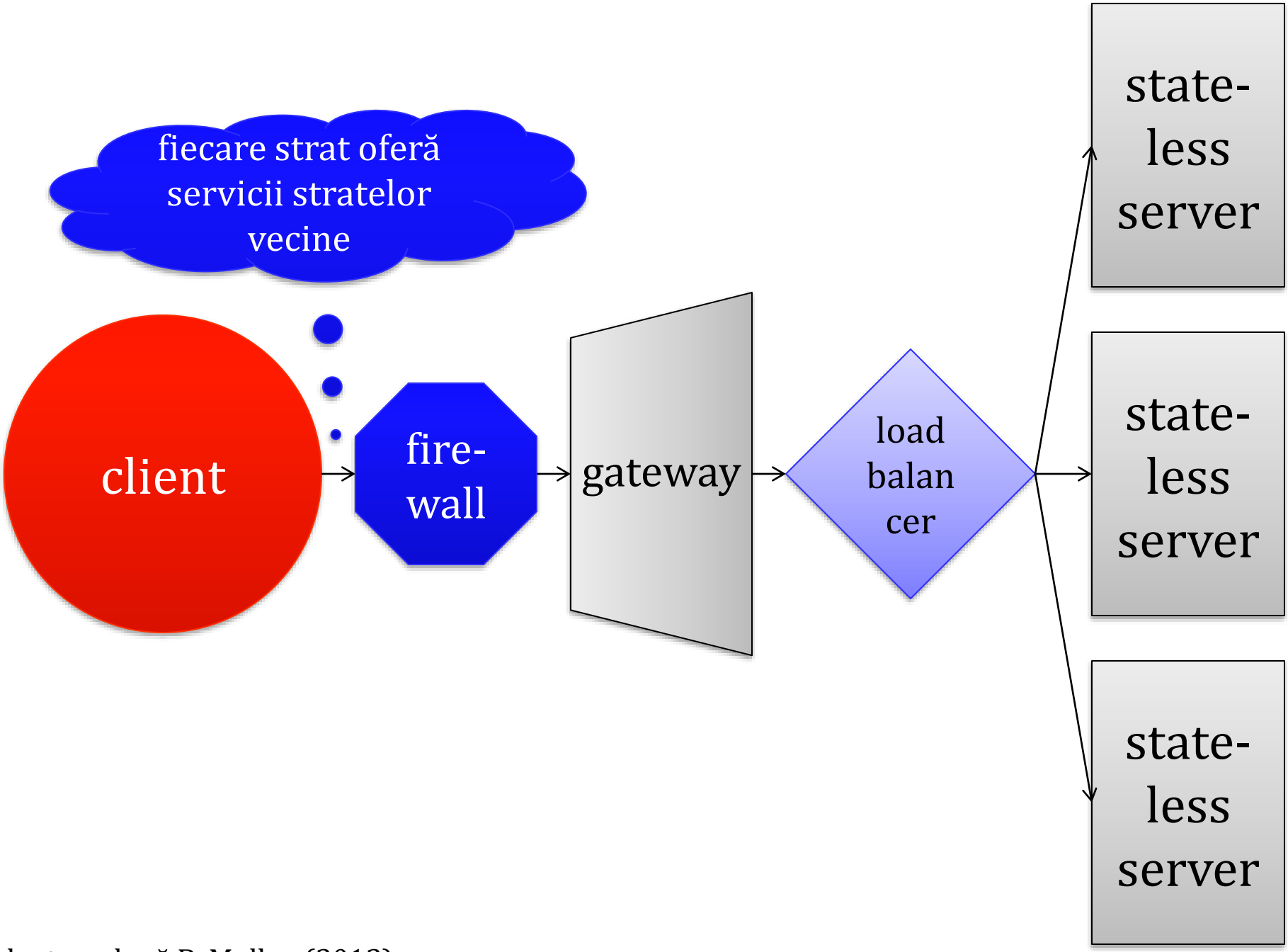
rest

Aplicația Web dezvoltată va fi stratificată

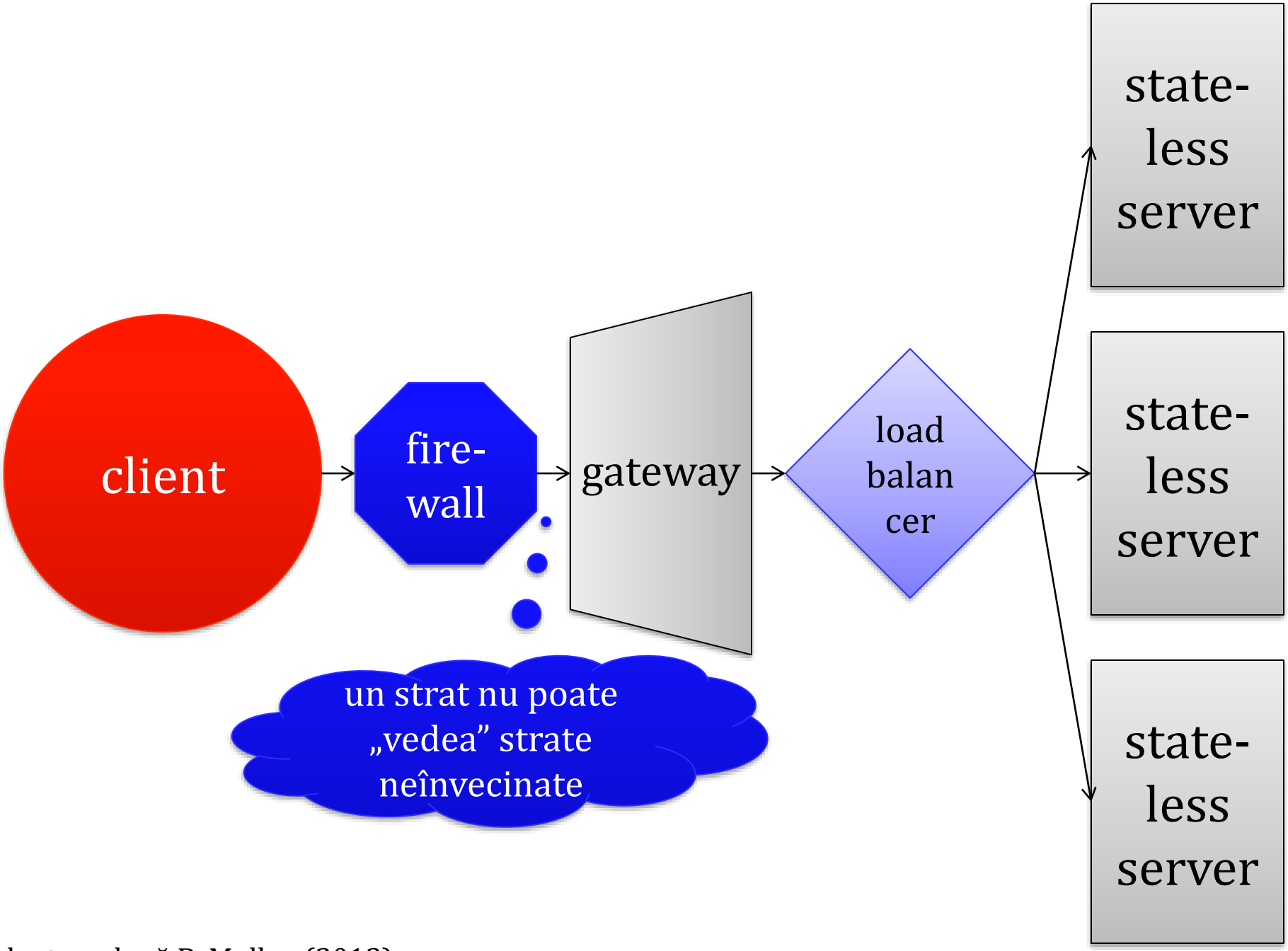
layered system



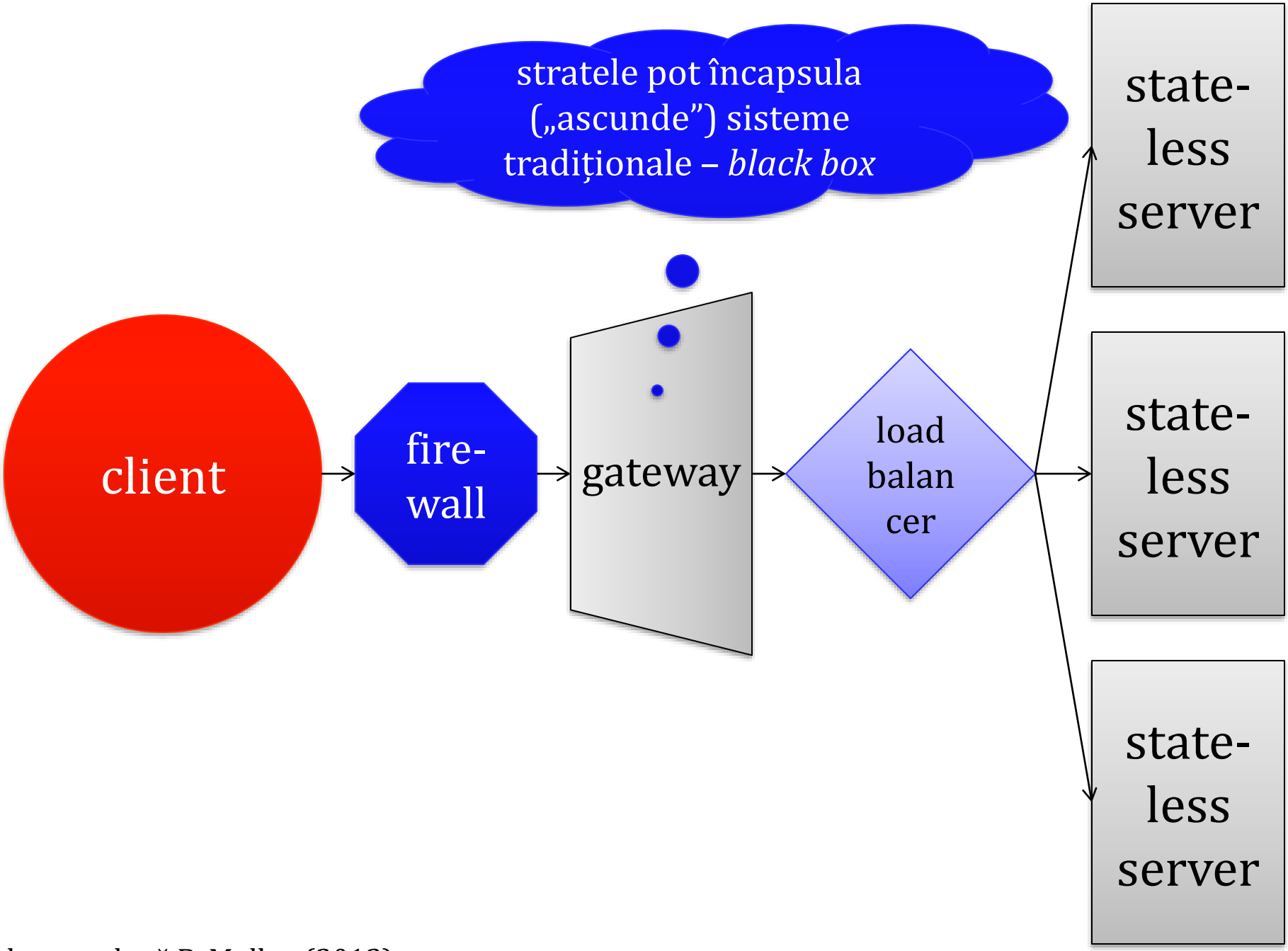
adaptare după B. Mulloy (2012)



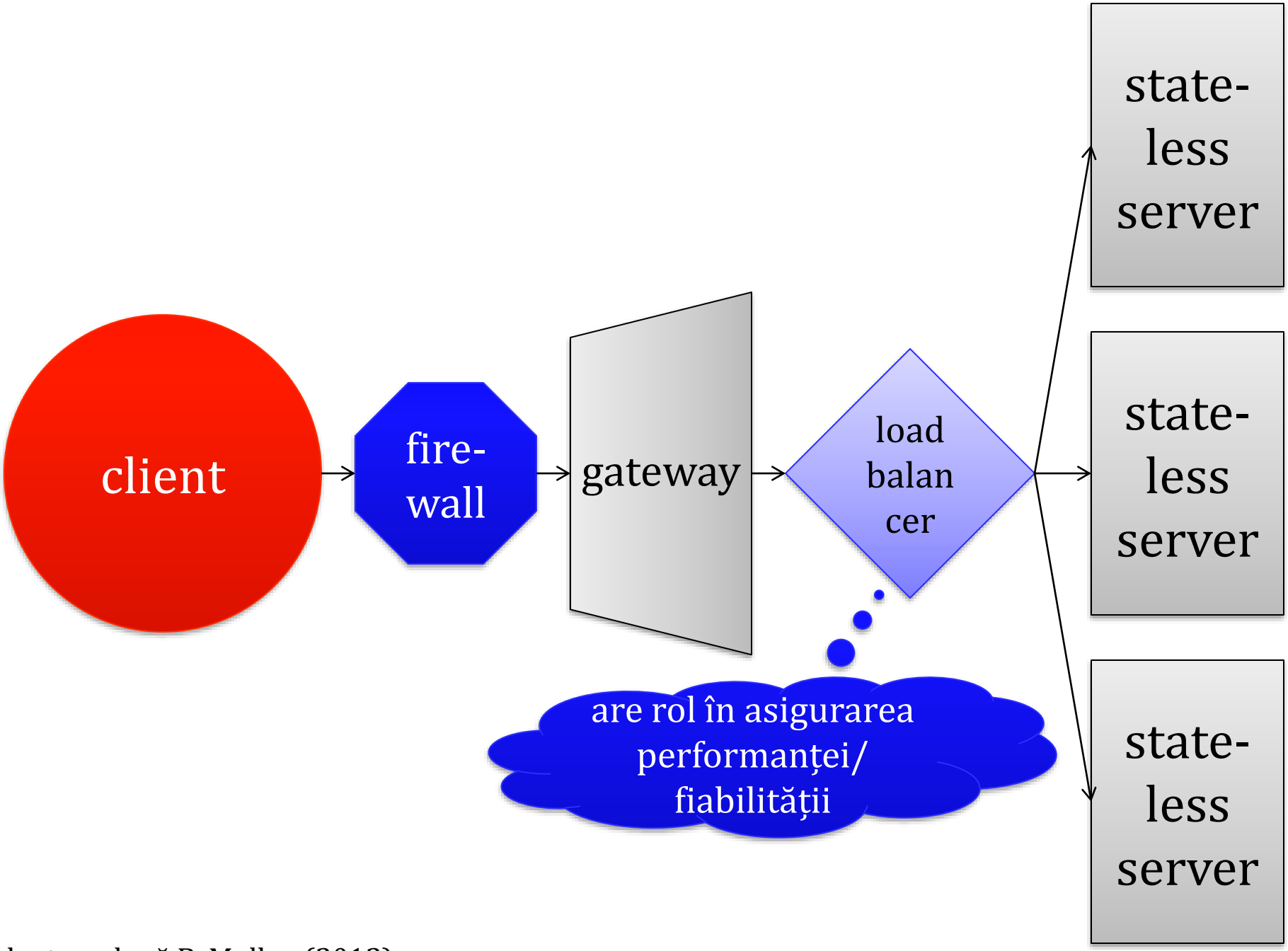
adaptare după B. Mulloy (2012)



adaptare după B. Mulloy (2012)



adaptare după B. Mulloy (2012)



adaptare după B. Mulloy (2012)

rest: exemplu

Implementarea unui magazin *on-line* oferind dulciuri

rest: exemplu

Implementarea unui magazin *on-line* oferind dulciuri

stilul „clasic” SOAP – conform RPC

operații privind produsele:
furnizeazăSortim(), adaugăSortim(), listeazăSortim(), cautăSortim()
operații ce vizează managementul utilizatorilor:
furnizeazăUtiliz(), adaugăUtiliz(), șterge Utiliz(), caută Utiliz(),...

rest: exemplu

Implementarea unui magazin *on-line* oferind dulciuri

stilul „nou” REST

tipuri de resurse (**Sortim** + **Utiliz**), identificate unic de URI

http://www.dulciuri.biz/sortim/portocale

rest: exemplu

Implementarea unui magazin *on-line* oferind dulciuri

stilul „nou” REST

tipuri de resurse (**Sortim** + **Utiliz**), identificate unic de URI

http://www.dulciuri.biz/sortim/portocale/albastre

URI intuitiv

“user/SEO friendly”

rest: exemplu

Serviciu pentru managementul adreselor Web favorite (*bookmark*-uri), cu posibilitatea atașării de termeni de conținut (*tag*-uri) și comentarii

social bookmarking

abordări similare: Delicious, Digg, Pocket, Reddit etc.

rest: exemplu

Serviciu pentru managementul adreselor Web favorite (*bookmark*-uri), cu posibilitatea atașării de termeni de conținut (*tag*-uri) și comentarii

funcționalitate de bază: listarea tuturor *bookmark*-urilor (eventual, filtrate după diverse criterii)

managementul *bookmark*-urilor:
adăugare, editare, ștergere, partajare

Resursa	URL	Metoda	Reprezentare
Bookmark	/bookmarks/{*hash*}	GET	application/bookmark+xml
Bookmark	/bookmarks/{*hash*}	PUT	application/bookmark+xml
Bookmark	/bookmarks/{*hash*}	DELETE	
Lista de adrese	/bookmarks	GET	application/atom+xml
Lista de utilizatori	/users	GET	application/atom+xml
Lista de *tag*-uri	/tags	GET	application/atom+xml
Pagina principală	/	GET	application/xml

GET /bookmarks
200 OK
Content-type: application/atom+xml

răspuns XML (în format Atom) oferit de serviciu

```
<?xml version="1.0"?>
<feed xmlns="http://www.w3.org/2005/Atom">
   <title>Bookmarks</title>
   <entry>
      <title>O resursă interesantă</title>
      <link
         href="/bookmarks/a211528f…bdcf"/>
      <summary>
         http://undeva.info/o-resursa-interesanta
      </summary>
   </entry>
   <!-- eventual, alte elemente <entry>… -->
</feed>
```

digest – hash (SHA-1, SHA-3,...)

obținerea *bookmark*-urilor

```
GET /bookmarks/a211528f…bdcf
200 OK
Content-type: application/bookmark+xml

<bookmark>
  <title>O resursă interesantă</title>
  <url>http://undeva.info/o-resursa-interesanta</url>
  <user href="/users/tux">tux</user>
  <tags>
    <tag href="/tags/interesting">interesting</tag>
    <tag href="/tags/penguin">penguin</tag>
  </tags>
</bookmark>
```

preluarea unui *bookmark*:
răspunsul XML oferit de serviciul Web

crearea
unui *bookmark*

```
POST /bookmarks
Content-type: application/bookmark+xml
...
201 Created
Location: /bookmarks/a211528f…bdcf
```

```
PUT /bookmarks/a211528f…bdcf
Content-type: application/bookmark+xml
...
200 OK
```

înlocuirea
unui *bookmark*

rest

Resursele se denumesc folosind URI-uri (URL-uri)

Reprezentările sunt interconectate prin URL-uri
"**Hypermedia as the engine of application state** *is a REST constraint.*"
(Roy Fielding, 2014)

Pot exista intermediari (*e.g.*, *proxy*, *cache*, *gateway*)
între clienți și resurse ▸ performanță, securitate,...

rest

Resursele se denumesc folosind URI-uri (URL-uri)

Reprezentările sunt interconectate prin URL-uri
"**Hypermedia as the engine of application state** *is a REST constraint.*"
(Roy Fielding, 2014)

Pot exista intermediari (*e.g.*, *proxy*, *cache*, *gateway*)
între clienți și resurse ▸ performanță, securitate,...

Transferul de date poate fi și asincron – Ajax *et al.*

într-un curs viitor

O resursă poate avea asociate reprezentări ce pot fi accesate/alterate via operații HTTP

CRUD – *Create, Retrieve, Update, Delete*

Operation	SQL	HTTP
Create	INSERT	PUT POST
Read (Retrieve)	SELECT	GET
Update (Modify)	UPDATE	PUT POST PATCH
Delete (Destroy)	DELETE	DELETE

caz concret: *framework*-ul LoopBack – loopback.io/doc/
operații↔REST↔model de implementare↔SQL

Model create, retrieve, update, and delete operations

When you connect a model to a persistent data source such as a database, it becomes a *connected model* with a full set of create, read, update, and delete operations from the PersistedModel class:

Operation	REST	LoopBack model method (Node API)*	Corresponding SQL Operation
Create	PUT /*modelName* POST /*modelName*	`create()`*	INSERT
Read (Retrieve)	GET /modelName?filter=...	`find()`*	SELECT
Update (Modify)	PUT /modelName	`updateAll()`*	UPDATE
Delete (Destroy)	DELETE /*modelName*/*modelID*	`destroyById()`*	DELETE

(*) Methods listed are just prominent examples; other methods may provide similar functionality; for example: `findById()`, `findOne()`, and `findOrCreate()`.

rest

Interacțiunea cu un serviciu Web dezvoltat în stilul REST se poate face via un API (*Application Programming Interface*)

vezi prelegerea următoare

Putem adopta o metodologie vizând dezvoltarea de servicii Web (API-uri) aliniate paradigmei REST?

avansat

rest: metodologie

Divizarea în resurse a setului de date ale problemei

clase tipice de resurse:

Utilizatori

Documente – alternative: **Fotografii**, **Produse**, **Software**,...

Metadate – *e.g.*, **Comentarii**, **Formate**, **Locații**, **Platforme** etc.

avansat

rest: metodologie

Divizarea datelor problemei în categorii

cazuri concrete:

Behance – **www.behance.net/dev/api/endpoints/**

Projects Users Fields Collections

Coinbase – **developers.coinbase.com/api/v2**

Accounts Addresses Transactions Deposits Prices Currencies

Discogs – **www.discogs.com/developers/**

Artists Releases Labels Versions Orders Stats

avansat

rest: metodologie

Divizarea datelor problemei în categorii

cazuri concrete:

StackExchange – **api.stackexchange.com**

Users Answers Badges Comments Questions Revisions Tags

Unsplash – **unsplash.com/documentation**

Users Photos Collections Stats

World of Warcraft – **develop.battle.net/documentation**

Characters Guilds Realms Auctions Items

avansat

rest: metodologie

„Numirea” prin URI a fiecărei resurse

exemplificări:

http://web.info/Utilizatori/tux

http://web.info/Documente/pinguini-cu-mere-albastre

avansat

rest: metodologie

„Numirea” prin URI a fiecărei resurse

cazuri concrete:
accesarea știrilor referitoare la un subiect de interes
https://www.reddit.com/r/programming/

acces la prezentările SlideShare ale utilizatorului busaco
https://www.slideshare.net/busaco/presentations

obținerea listei celor ce urmăresc un utilizator autentificat
https://twitter.com/followers

avansat

rest: metodologie

Organizarea resurselor

aceste resurse (*object instances*) pot fi organizate în colecții (*collections*) sau depozite (*stores*)

a se consulta D. Denicola, *Creating Truly RESTful APIs* (2013)
www.slideshare.net/domenicdenicola/creating-truly-res-tful-apis

avansat

rest: metodologie

Organizarea resurselor

colecție

catalog de resurse gestionate de **server**
clienții pot propune alterarea colecției
serverul decide care-i rezultatul unei operații

exemple (GitHub): **/orgs/openstack/repos**, **/orgs/openstack/events**

avansat

rest: metodologie

Organizarea resurselor

depozit

„rezervă” de resurse gestionată de **client**
(inclusiv filtrare, sortare, paginare, accesare meta-date,...)

exemplu (GitHub): **/users/openstack/repos?page=2&per_page=3**

avansat

rest: metodologie

Proiectarea reprezentării(lor) *acceptate*
ce pot fi trimise de aplicația client
și reprezentării(lor) *întoarse* spre client

avansat

rest: metodologie

Proiectarea reprezentării(lor) *acceptate*
ce pot fi trimise de aplicația client
și reprezentării(lor) *întoarse* spre client

de considerat formatele standard, comune:
CSV – *Comma Separated Values*
JSON(-LD) – *JavaScript Object Notation* (-*Linked Data*)
XML – *Extensible Markup Language*

avansat

rest: metodologie

Integrarea resurselor
via legături hipermedia + formulare

exemplificare (GitHub):
"All resources may have one or more *_url *properties linking to other resources. These are meant to provide explicit URLs so that proper API clients don't need to construct URLs on their own."*
developer.github.com/v3/#hypermedia

avansat

rest: metodologie

Crearea de studii de caz

specificarea condițiilor de eroare și/sau de excepție

eventual, aspecte privind controlul versiunilor API-ului

StrongLoop API Explorer

Token Not Set accessToken

avansat

Users

Show/Hide | List Operations | Expand Operations | Raw

POST	/Users/login	Login a user with username/email and password
POST	/Users/logout	Logout a user with access token
GET	/Users/confirm	Confirm a user registration with email verification token
POST	/Users/reset	Reset password for a user with email
GET	/Users/{id}/accessTokens/{fk}	Find a related item by id for accessTokens
DELETE	/Users/{id}/accessTokens/{f	
PUT	/Users/{id}/accessTokens/{f	
GET	/Users/{id}/accessTokens	
POST	/Users/{id}/accessTokens	Creates a new instance in accessTokens of this model.
DELETE	/Users/{id}/accessTokens	Deletes all accessTokens of this model.
GET	/Users/{id}/accessTokens/count	Counts accessTokens of User.
POST	/Users	Create a new instance of the model and persist it into the data source
PUT	/Users	Update an existing model instance or insert a new one into the data source
GET	/Users	Find all instances of the model matched by filter from the data source
GET	/Users/{id}/exists	Check whether a model instance exists in the data source
HEAD	/Users/{id}	Check whether a model instance exists in the data source
GET	/Users/{id}	Find a model instance by id from the data source
DELETE	/Users/{id}	Delete a model instance by id from the data source
PUT	/Users/{id}	Update attributes for a model instance and persist it into the data source

StrongLoop API

operații cu resurse specifice – aici **Users**

avansat

POST /Users Create a new instance of th ource

PUT /Users Update an existing model instance or insert a new one into the data source

Response Class

Model Model Schema

```
{
  "realm": "",
  "username": "",
  "credentials": "object",
  "challenges": "object",
  "email": "",
  "emailVerified": false,
  "verificationToken": "",
  "status": "",
  "created": "",
  "lastUpdated": "",
```

StrongLoop API

testarea interactivă a API-ului

strongloop.com/strongblog/node-js-rest-api-openshift-redhat/

Response Content Type application/json

Parameters

Parameter	Value	Description	Parameter Type	Data Type

data	Parameter content type: application/json (application/json, application/x-www-form-urlencoded)	Model instance data	body	Model Model Schema
				User {
				realm (string, *optional*),
				username (string, *optional*),
				credentials (object, *optional*),
				challenges (object, *optional*),
				email (string),
				emailVerified (boolean, *optional*),
				verificationToken (string, *optional*),
				status (string, *optional*),
				created (string, *optional*),
				lastUpdated (string, *optional*),
				id (number, *optional*)
				}

Try it out!

(în loc de) pauză

Aspecte practice de interes pentru dezvoltatori?

avansat

rest: privire pragmatică

URL-urile desemnând resurse (concepte) de interes trebuie să fie simple și intuitive

utilizarea substantivelor pentru fiecare „lucru” (entitate)

avansat

rest: privire pragmatică

URL-urile desemnând resurse (concepte) de interes trebuie să fie simple și intuitive

colecții de resurse – uzual, la plural

/students

avansat

rest: privire pragmatică

URL-urile desemnând resurse (concepte) de interes trebuie să fie simple și intuitive

colecții de resurse – uzual, la plural
/students

identificatori unici pentru membrii unei colecții
/students/tuxy (concret) *vs.* **/students/69** (abstract)

avansat

rest: privire pragmatică

URL-urile desemnând resurse (concepte) de interes trebuie să fie simple și intuitive

structura ierarhică a URL-urilor reprezintă ierarhia resurselor din cadrul domeniului modelat al aplicației

exemplu (Discogs):

/users/{name}/collection/folders/{f_id}/releases/{r_id}/instances/{i_id}

avansat

rest: privire pragmatică

Folosirea verbelor (metodelor) HTTP pentru efectuarea de operații asupra unor (colecții de) resurse

resursa (URI)	POST (creează)	GET (accesează)	PUT (actualizează)	DELETE (șterge)
/students	creează un student nou	listează studenții existenți	actualizează un set de studenți	șterge toți studenții
/students/69 (un URL deja existent)	eroare ☹	oferă date despre student	dacă există, actualizează, altfel eroare	șterge studentul respectiv

avansat

rest: privire pragmatică

Raportarea erorilor

folosirea codurilor de stare HTTP – **httpstatuses.com**

exemple tipice:
200 *OK*, 204 *No Content*, 206 *Partial Content*
303 *See Other*, 304 *Not Modified*
400 *Bad Request*, 401 *Unauthorized*, 403 *Forbidden*,
404 *Not Found*, 405 *Method Not Allowed*
500 *Internal Server Error*, 503 *Service Unavailable*

avansat

rest: privire pragmatică

Raportarea erorilor

mesajele oferite trebuie să includă informații utile

avansat

rest: privire pragmatică

Raportarea erorilor

mesajele oferite trebuie să includă informații utile

exemplu:
GitHub – cod de stare HTTP oferit: 404

```
{
  "message": "Not Found",
  "documentation_url": "https://developer.github.com/v3"
}
```

orice reprezentare oferită include minim un URL

avansat

rest: privire pragmatică

Controlul versiunilor API-ului dezvoltat

"*Never release an API without a version and make the version mandatory.*" (Mulloy, 2012)

considerații de interes în articolul J. Curry, *Introduction to API Versioning Best Practices* (2017): **nordicapis.com/introduction-to-api-versioning-best-practices/**

avansat

rest: privire pragmatică

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

soluții uzuale:
în antetul cererii HTTP
în cadrul URL-ului

avansat

rest: privire pragmatică

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

în antetul cererii HTTP

Accept: **application/vnd.heroku+json**; **version=3**
Accept: **application/vnd.discogs.v2.html+json**

în unele cazuri, folosind un antet propriu: X-API-Version

avansat

rest: privire pragmatică

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

în cadrul URL-ului – eventual, ca parametru

api.yelp.com/v3/businesses/

ec2.amazonaws.com/?Action=RunInstances&Version=2016-11-15

avansat

rest: privire pragmatică

Controlul versiunilor API-ului dezvoltat

specificarea versiunii

continuous versioning

acces via același URI, indiferent de versiunea curentă

- practica preferată (“*Cool URIs don’t change*”)

nordicapis.com/continuous-versioning-strategy-for-internal-apis/

avansat

rest: privire pragmatică

Paginarea și oferirea de răspunsuri parțiale

practică uzuală pentru paginare: **page** și **per_page**

/students/projects?page=2&per_page=5

avansat

rest: privire pragmatică

Paginarea și oferirea de răspunsuri parțiale

practică uzuală pentru paginare: **page** și **per_page**

/students/projects?page=2&per_page=5

alternativ, se pot folosi parametri precum **limit** și **offset**

/students/projects?limit=33&offset=54

avansat

rest: privire pragmatică

Paginarea și oferirea de răspunsuri parțiale

filtrele opționale pot fi delimitate de virgulă

/students?fields=name,age,year,email

avansat

rest: privire pragmatică

Eterogenitatea formatelor reprezentărilor întoarse

indicarea formatului în URL via un parametru opțional

?fmt=json (MusicBrainz)

avansat

rest: privire pragmatică

Eterogenitatea formatelor reprezentărilor întoarse

specificarea formatului acceptat în antetul cererii HTTP

Accept: application/json (Digg)

avansat

rest: privire pragmatică

Eterogenitatea formatelor reprezentărilor întoarse

precizarea formatului în numele resursei solicitate

/venue.json (Foursquare)

avansat

Movie Reviews API

DOWNLOAD SPEC | AUTHORIZE

Try this API

Request parameters

type

picks

offset

integer

✓ by-opening-date
by-publication-date
by-title

Credentials

apikey

EXECUTE

cURL | HTTP

```
GET https://api.nytimes.com/svc/movies/v2/reviews/picks.json?order=by-openi

Accept: application/json
```

/movies/v2/reviews/picks.json?order=...

application/json | Raw HTTP Response

```
{
  "display_title": "Parasite",
  "mpaa_rating": "R",
  "critics_pick": 1,
  "byline": "Manohla Dargis",
  "headline": "'Parasite' Review: The Lower Depths Rise With a Vengeance",
  "summary_short": "In Bong Joon Ho's new film, a destitute family occupies
  "publication_date": "2019-10-10",
```

răspunsul în format JSON

interogări interactive asupra serviciului de recenzie de filme (Movie Reviews) oferit de *The New York Times*

developer.nytimes.com

avansat

rest: privire pragmatică

Utilizarea subdomeniilor pentru API-uri diferite ale aceluiași ofertant de servicii

exemplificare:

search.twitter.com

stream.twitter.com

api.twitter.com

Cum pot fi accesate reprezentări de resurse Web prin REST?

rest: implementare

Navigatoarele Web actuale

nu necesită un API distinct pentru acces via HTTP

disponibilitate pe orice platformă

suport pentru REST prin obiectul **XMLHttpRequest** (Ajax), **Fetch API** (HTML5) ori **WebSocket API** (HTML5)

rest: implementare – biblioteci

Akka HTTP (Java, Scala): **doc.akka.io/docs/akka-http/**
AlamoFire (Swift): **github.com/Alamofire/Alamofire**
Apache HttpComponents (Java): **hc.apache.org**
Axios (Node.js): **axios-http.com**
Guzzle (PHP 7+): **docs.guzzlephp.org/en/stable/**
haskage (Haskell): **hackage.haskell.org/package/HTTP**
http.client (Python 3): **docs.python.org/3/library/http.client.html**
http.rb (Ruby): **github.com/httprb/http/wiki**
HTTP Toolkit (soluție multi-platformă): **httptoolkit.tech**
khttp (Kotlin + Java): **khttp.readthedocs.io**

rest: implementare – biblioteci

HATEOS (PHP): **github.com/willdurand/Hateoas**
Httpful (PHP): **phphttpclient.com**
Hyper (Rust): **hyper.rs**
LibcURL (C; portări Perl, PHP, Ruby,...): **curl.haxx.se/libcurl/**
LibHTTP (C): **www.libhttp.org**
LWP (Perl): **metacpan.org/release/libwww-perl**
node-fetch (Node.js): **github.com/node-fetch/node-fetch**
Requests (PHP): **requests.ryanmccue.info**
restc-cpp (C++): **github.com/jgaa/restc-cpp**
RestSharp (.NET): **restsharp.dev**

rest: implementare – exemplul 1

Invocarea unui serviciu Web de prescurtare a URL-urilor – **https://is.gd/**

un nou URL prescurtat va fi creat folosind adresa
https://is.gd/create.php?format=xml&url=*adresaWeb*

alte formate:
web (HTML), json,
simple (text)

Cererea HTTP ce invocă serviciul Web:

```
GET /create.php?format=xml&url=profs.info.uaic.ro/~busaco HTTP/1.1
Host: is.gd
```

Cererea HTTP ce invocă serviciul Web:

```
GET /create.php?format=xml&url=profs.info.uaic.ro/~busaco HTTP/1.1
Host: is.gd
```

Răspunsul obținut, transmis de serverul Web:

```
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 20 Apr 2021 14:32:07 GMT+2
Content-Type: text/xml;charset=UTF-8

<?xml version="1.0" encoding="UTF-8" ?>
<output>
    <shorturl>https://is.gd/K3oomj</shorturl>
</output>
```

reprezentare POX
(*Plain Old XML*)

apelarea serviciului Web via LibcURL (PHP)

```
$c = curl_init (); // inițializăm LibcURL
// stabilim URL-ul serviciului Web invocat
curl_setopt ($c, CURLOPT_URL,
        'https://is.gd/create.php?format=xml&url=profs.info.uaic.ro/~busaco');
// rezultatul cererii va fi disponibil ca șir de caractere
curl_setopt ($c, CURLOPT_RETURNTRANSFER, true);
// nu verificăm certificatul digital utilizat pentru transferul datelor cu HTTPS
curl_setopt ($c, CURLOPT_SSL_VERIFYPEER, false);
// preluăm reprezentarea oferită de server (aici, un document XML)
$res = curl_exec ($c); // implicit se recurge la metoda GET
curl_close ($c);          // închidem conexiunea cURL

$doc = new DOMDocument (); // procesăm rezultatul via DOM
$doc->loadXML ($res);
// preluăm conținutul elementului <shorturl>
$urls = $doc->getElementsByTagName ('shorturl');
foreach ($urls as $url) {
  echo '<p>New short URL is ' . $url->nodeValue . '</p>';
}
```

rest: implementare – exemplul 1

Invoking Web service from `https://is.gd/create.php?format=xml&url=profs.info.uaic.ro/~busaco`

Server response:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<output><shorturl>https://is.gd/K3oomj</shorturl></output>
```

New short URL is https://is.gd/K3oomj

rest: implementare – exemplul 2

Obținerea unei imagini cu feline corespunzătoare unui cod de stare HTTP dat

ca și în cazul anterior, serviciul Web e disponibil liber, fără autentificare, dar nu respectă toate principiile REST (nu există URL-uri spre alte resurse) – *i.e.* nu e *RESTful*

https://http.cat/

```
define ('URL', 'https://http.cat/301');

$c = curl_init (URL); // inițializăm LibcURL, indicând URL-ul serviciului

$opt = [ // tabloul opțiunilor de realizare a cererii HTTP
  // datele vor fi disponibile ca șir de caractere
  CURLOPT_RETURNTRANSFER        => true,
  // nu verificăm certificatul digital
  CURLOPT_SSL_VERIFYPEER        => false,
  // timp de așteptare (în secunde) a stabilirii conexiunii
  CURLOPT_CONNECTTIMEOUT        => 10,
  // timp de așteptare (în secunde) a răspunsului
  CURLOPT_TIMEOUT               => 10,
  // codurile 4XX vor conduce la eroare
  CURLOPT_FAILONERROR           => true,
  // nu se acceptă redirecționări
  CURLOPT_FOLLOWLOCATION        => false ];

curl_setopt_array ($c, $opt);
$res = curl_exec ($c);
```

vezi exemplul din arhivă

```
// codul de stare HTTP întors de serverul serviciului Web
$codHTTP = curl_getinfo ($c, CURLINFO_RESPONSE_CODE);
// tipul conținutului oferit de serviciu
$tip = curl_getinfo ($c, CURLINFO_CONTENT_TYPE);

// am primit cu succes o imagine în format JPEG?
if ($codHTTP == 200 && $tip == 'image/jpeg') {
  header ('Content-Type: ' . $tip); // trimitem tipul MIME corespunzător
  echo $res; // afișăm reprezentarea resursei obținute (aici, imaginea JPEG)
} else {
  // trimitem codul de stare întors de serviciu
  http_response_code ($codHTTP);
  header ('Content-Type: text/plain');
  echo 'Status code: ' . $codHTTP;

}

curl_close ($c);          // închidem conexiunea cu serverul Web
```

vezi exemplul din arhivă

rest: implementare – exemplul 2

301

Moved Permanently

conținutul redat + detalii privind transferul datelor

Status	Method	File	Protocol	Type	Transferred	Size
200	GET	rest-cat.php	HTTP/1.1	jpeg	42.35 KB	42.11 KB
200	GET	favicon.ico	HTTP/1.1	x-i...	30.48 KB	30.17 KB

2 requests | 72.28 KB / 72.83 KB transferred | Finish: 635 ms | DOM

Headers | Cookies | Params | Response | Timings

Request URL: `http://localhost/php/rest-cat.php`

Request Method: `GET`

Remote Address: `127.0.0.1:80`

Status Code: 200 OK

Version: `HTTP/1.1`

Edit and Resend

Filter Headers

Response Headers (247 B) Raw Headers

Connection: Keep-Alive

Content-Type: image/jpeg

Date: Tue, 14 Apr 2020 11:17:40 GMT

Keep-Alive: timeout=5, max=100

Server: Apache/2.4.41 (Win64) OpenSSL/1.1.1c PHP/7.4.3

Transfer-Encoding: chunked

X-Powered-By: PHP/7.4.3

rest: implementare – exemplul 3

Accesarea datelor publice despre Pokémon

colecții de resurse:
Pokémon, Locations, Contests, Games, Moves,...

fiecare categorie de resurse are proprietăți specifice
e.g., orice instanță de Locations include id, name, region etc.

detalii la **pokeapi.co**

https://pokeapi.co/api/v2/ pokemon/ditto

Need a hint? Try pokemon/ditto, pokemon/1, type/3, ability/4, or pokemon?limit=100&offset=200.

Direct link to results: https://pokeapi.co/api/v2/pokemon/ditto

Resource for ditto


```
▼ abilities: [] 2 items
  ▼ 0: {} 3 keys
    ▼ ability: {} 2 keys
        name: "limber"
        url: "https://pokeapi.co/api/v2/ability/7/"
      is_hidden: false
      slot: 1
  ▼ 1: {} 3 keys
    ▼ ability: {} 2 keys
        name: "imposter"
        url: "https://pokeapi.co/api/v2/ab
      is_hidden: true
```

```
  effect_changes: [] 0 items
▼ effect_entries: [] 2 items
  ▶ 0: {} 3 keys
  ▼ 1: {} 3 keys
      effect: "This Pokémon cannot be paralyzed. If a Pokémon is paralyzed and
        acquires this ability, its paralysis is healed; this includes when reg
        aining a lost ability upon leaving battle."
    ▶ language: {} 2 keys
      short_effect: "Prevents paralysis."
▶ flavor_text_entries: [] 45 items
▼ generation: {} 2 keys
    name: "generation-iii"
    url: "https://pokeapi.co/api/v2/generation/3/"
  id: 7
  is_main_series: true
  name: "limber"
▶ names: [] 10 items
```


răspuns disponibil în format JSON

rest: implementare – exemplul 4

Preluarea datelor despre artefacte culturale oferite de *Metropolitan Museum of Art*

metmuseum.github.io

disponibile liber sub licența *Creative Commons Zero*

Objects – colecții de resurse
Object – include (meta-)date de interes
Departments – lista departamentelor muzeului

rest: implementare – exemplul 4

Preluarea datelor despre artefacte culturale oferite de *Metropolitan Museum of Art*

metmuseum.github.io

API-ul REST oferit poate fi accesat fără autentificare folosind domeniul **collectionapi.metmuseum.org**
rezultatele interogărilor sunt disponibile în format JSON

parte componentă a inițiativei Google Arts & Culture

artsandculture.google.com

rest: implementare – exemplul 4

Preluarea datelor despre artefacte culturale oferite de *Metropolitan Museum of Art*

pasul #1:
căutarea – cerere GET – unor resurse de interes

collectionapi.metmuseum.org/public/collection/v1/search?q=Romania

rest: implementare – exemplul 4

Preluarea datelor despre artefacte culturale oferite de *Metropolitan Museum of Art*

pasul #1:
căutarea – cerere GET – unor resurse de interes

se obține o colecție JSON de identificatori – objectIDs
{ "total": 50, "objectIDs": [460405, 29327, 98431,…, 159148] }

rest: implementare – exemplul 4

Preluarea datelor despre artefacte culturale oferite de *Metropolitan Museum of Art*

pasul #2:
accesul – cu GET – la datele vizând o resursă specifică

collectionapi.metmuseum.org/public/collection/v1/objects/98431

objectID

rest: implementare – exemplul 4

Preluarea datelor despre artefacte culturale oferite de *Metropolitan Museum of Art*

pasul #2:
accesul – cu GET – la datele vizând o resursă specifică

rezultatul reprezintă un obiect JSON încapsulând diverse (meta-)date furnizate de proprietăți: imagini ale artefactului (primaryImage, additionalImages), proveniența (country, state, region, city), clasificarea (classification),...

câmpurile-antet ale cererii HTTP:

Accept: **application/json,application/xml;q=0.9,*/*;q=0.8**
Accept-Encoding: **gzip, deflate, br**
Accept-Language: **en,en-GB;q=0.5**
Connection: **keep-alive**
Host: **collectionapi.metmuseum.org**
User-Agent: **Mozilla/5.0 … Gecko/20100101 Firefox/87.0**

câmpurile-antet ale răspunsului furnizat de API:

Access-Control-Allow-Origin: *****
Content-Encoding: **gzip**
Content-Type: **application/json; charset=UTF-8**
Date: **Tue, 20 Apr 2021 10:15:32 GMT**
Transfer-Encoding: **chunked**
X-CDN: **Imperva**

(meta-)date oferite de serverul Web

```
objectID:            98431
isHighlight:         false
accessionNumber:     "09.50.2690"
accessionYear:       "1909"
isPublicDomain:      true
primaryImage:        "https://images.metmuseum…/original/09.50.2690.jpg"
primaryImageSmall:   "https://images.metmuseum…web-large/09.50.2690.jpg"
additionalImages:
  0:                 "https://images.metmuseum.org/CRDImages/ci/original/14297.jpg"
constituents:        null
department:          "Costume Institute"
objectName:          "Apron"
title:               "Apron"
culture:             "Romanian"
```


răspunsul JSON – incluzând URL-uri spre reprezentări grafice – oferit de serviciul Web apelat

rest: implementare

Biblioteci/API-uri implementând HTTP

permit dezvoltarea de aplicații *desktop*, mobile etc.

suport pentru crearea de aplicatii hibride (*mash-up*-uri) la nivel de server

nu funcționează în navigatorul Web

atenție la problemele de securitate ce pot apărea!

avansat

rest: dezvoltare – exemplificări

Micro-framework-uri PHP:

Fat-Free – **fatfreeframework.com**

Lumen – **lumen.laravel.com**

phprest – **github.com/phprest/phprest**

Slim – **www.slimframework.com**

Siler – **siler.leocavalcante.dev**

specificația JAX-RS (*Jakarta RESTful Web Services*): JSR-370

projects.eclipse.org/projects/ee4j.jaxrs

implementări: Apache CXF, Eclipse Jersey, EverRest, Jello

avansat

rest: dezvoltare – exemplificări

ASP.NET MVC + Web API (C# *et al.*): **www.asp.net/web-api**

Django REST framework, hug, restless (Python)

Express, LoopBack, Restify, Sails, Superagent (Node.js)
www.npmjs.com/search?q=REST

Grape, RESTRack, Ruby on Rails (Ruby)

pREST (*PostgreSQL RESTful Engine*): **docs.prestd.com**

a se consulta și **github.com/marmelab/awesome-rest**

rest: dezvoltare

Servicii publice ce pot fi consumate via REST – exemple:
Amazon, Coronavirus APIs, eBay, Facebook, FitBit, GitLab, Google, ISBNdb, LinkedIn, Microsoft, MusicBrainz, Open Banking Tracker, OpenStreetMap, PayPal, PostNL, Quora, Tumblr, Vimeo, Wikidata,…

de explorat situl ProgrammableWeb
www.programmableweb.com/category/all/apis
+
o listă a API-urilor disponibile pentru public
github.com/abhishekbanthia/Public-APIs

avansat

Groupon Goods

Sort by Relevance

Categories

- Auto & Home Improvement (34972)
- Baby, Kids & Toys (22628)
- Electronics (31391)
- Entertainment (6637)
- For the Home (104773)
- Grocery & Household (8011)
- Health & Beauty (60546)
- Jewelry & Watches (49666)
- Men's Fashion (41195)
- Pet Supplies (8576)
- Sports & Outdoors (36464)
- Personalized Items (857)
- Women's Fashion (43508)

Discount Types

- [x] All
- [ ] Clearance

Microfiber Luxury Home Ultra Soft Sheet Set (6-Piece)

This sheet set is made of an extra soft microfiber material which resists stains, wrinkles, and pilling, for crisp look and cozy feel

100,000+ bought

★★★★½ (13,050)

~~$99.99~~ **$12.99**

Sale Ends 5/12

View Deal

studiu de caz: **Groupon**

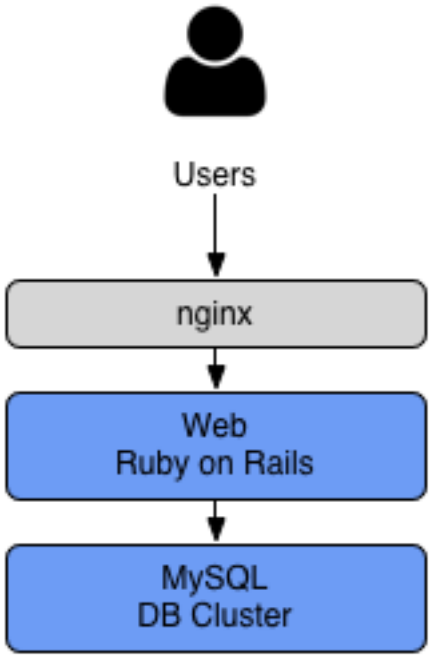
avansat

Scop: prezentarea de oferte de bunuri de consum

de la o arhitectură monolitică
la una adoptând servicii (API-uri) REST

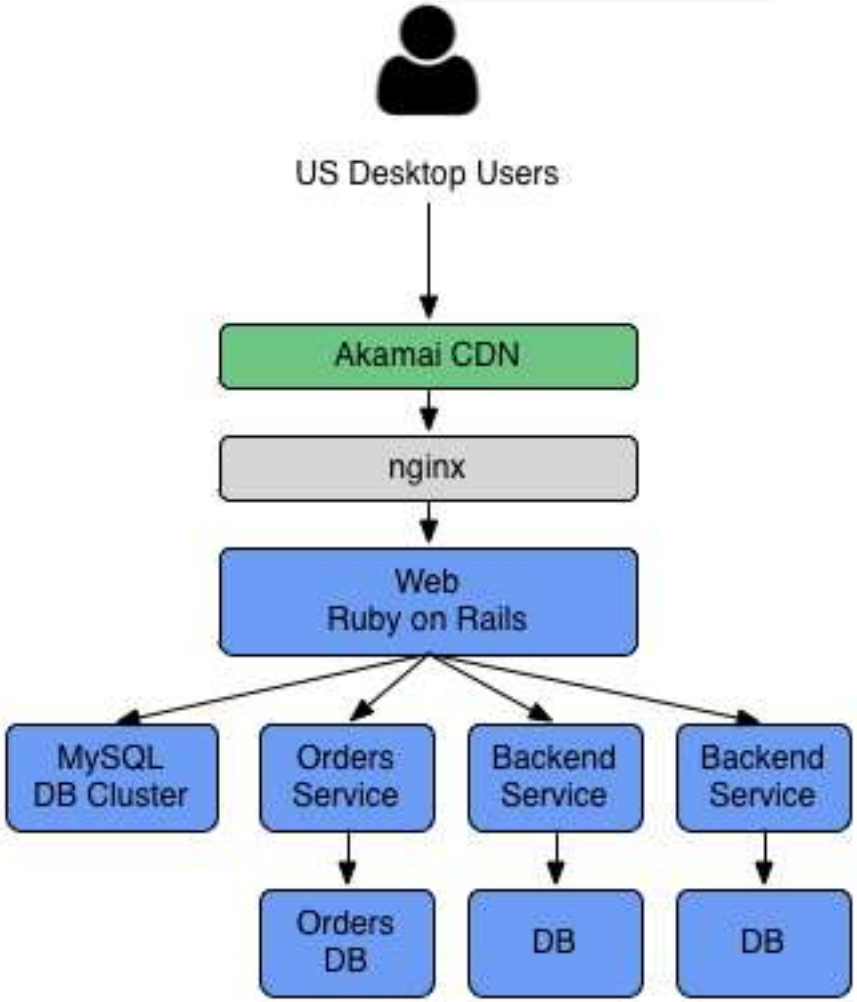
engineering.groupon.com/2013/misc/i-tier-dismantling-the-monoliths/

avansat



arhitectură inițială

MVC tradițional



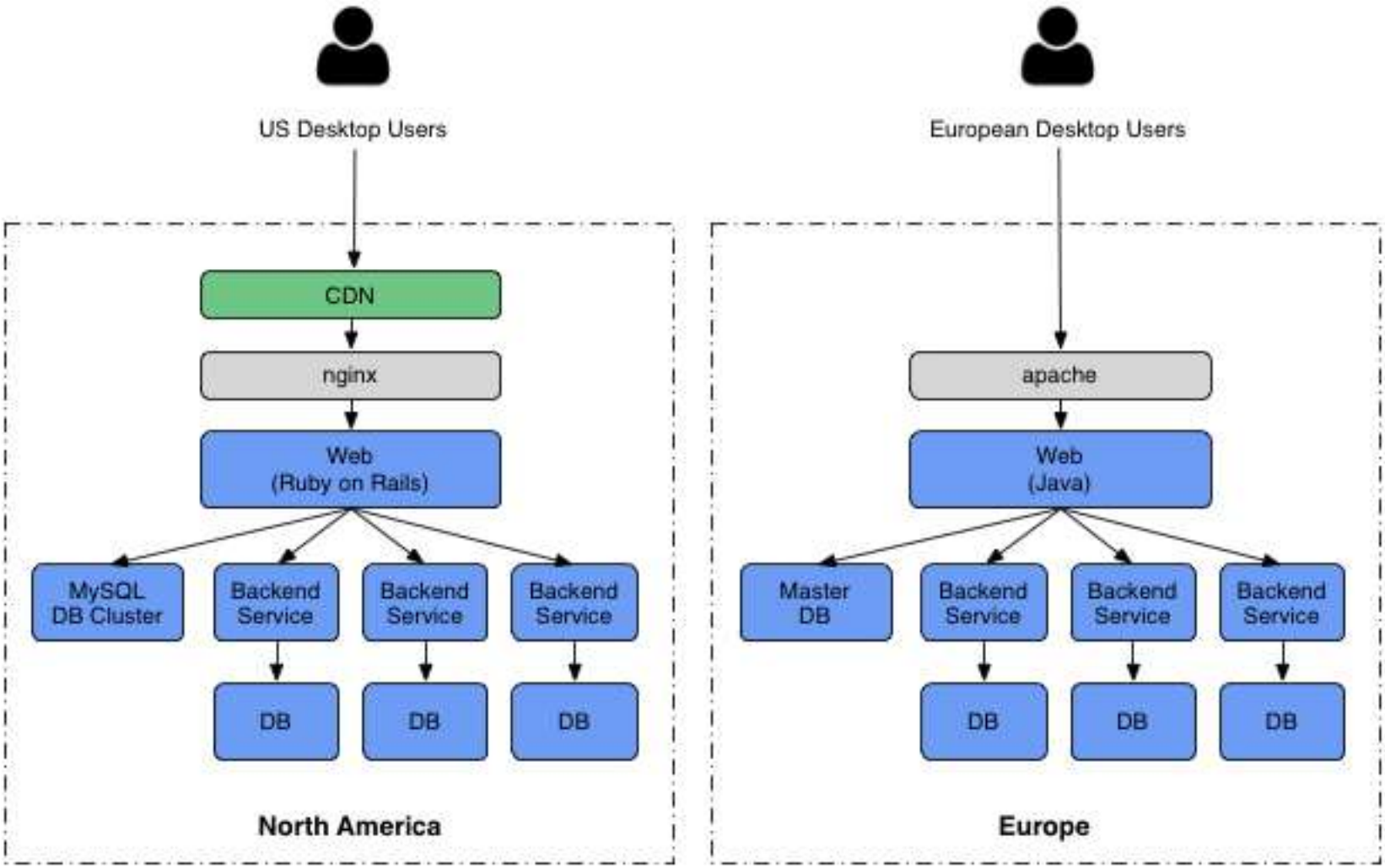
asigurarea performanței

scalabilitate cu CDN

(*Content Distribution Network*)

și servicii de acces la date

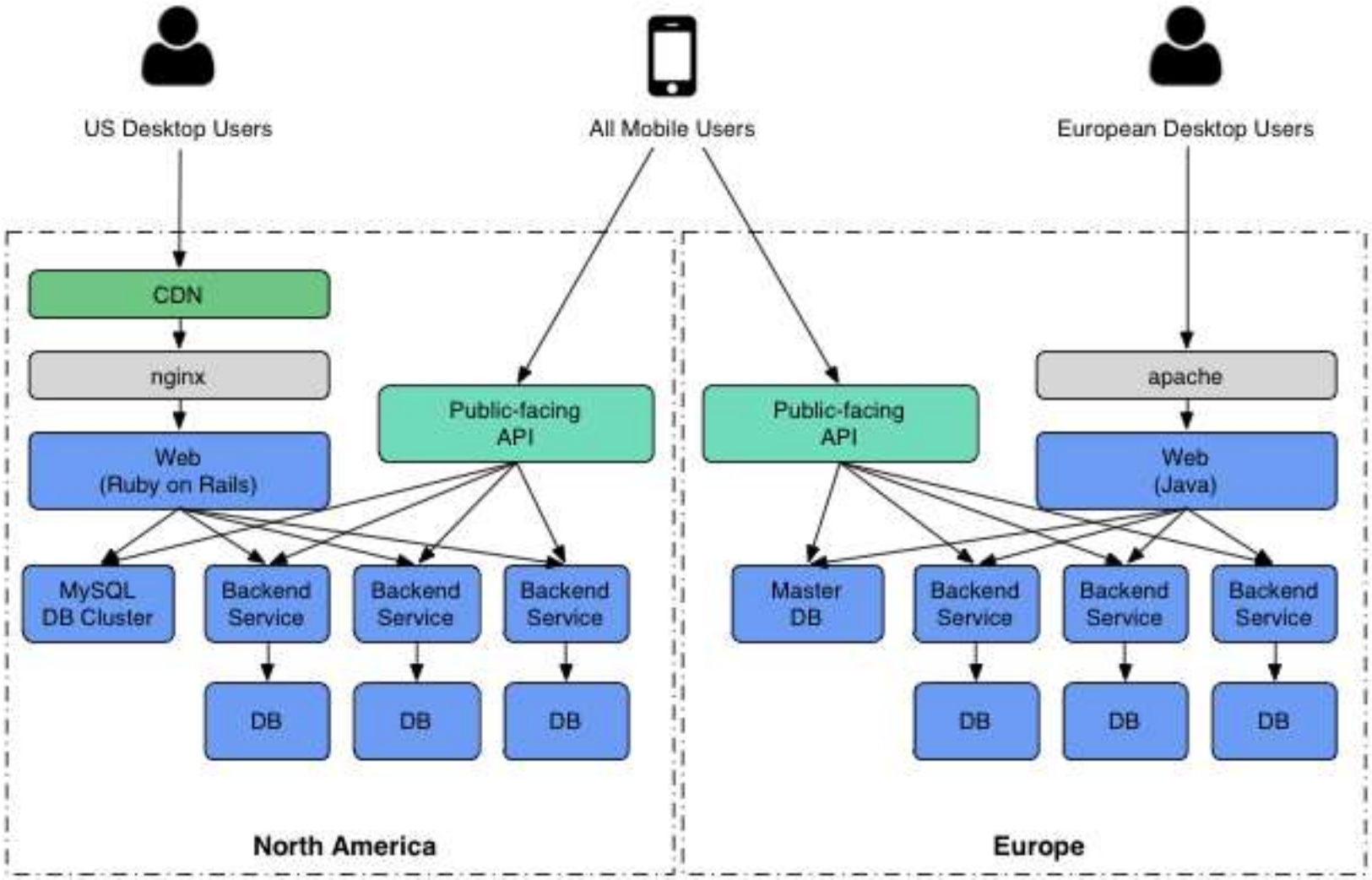
avansat



arhitectură eterogenă

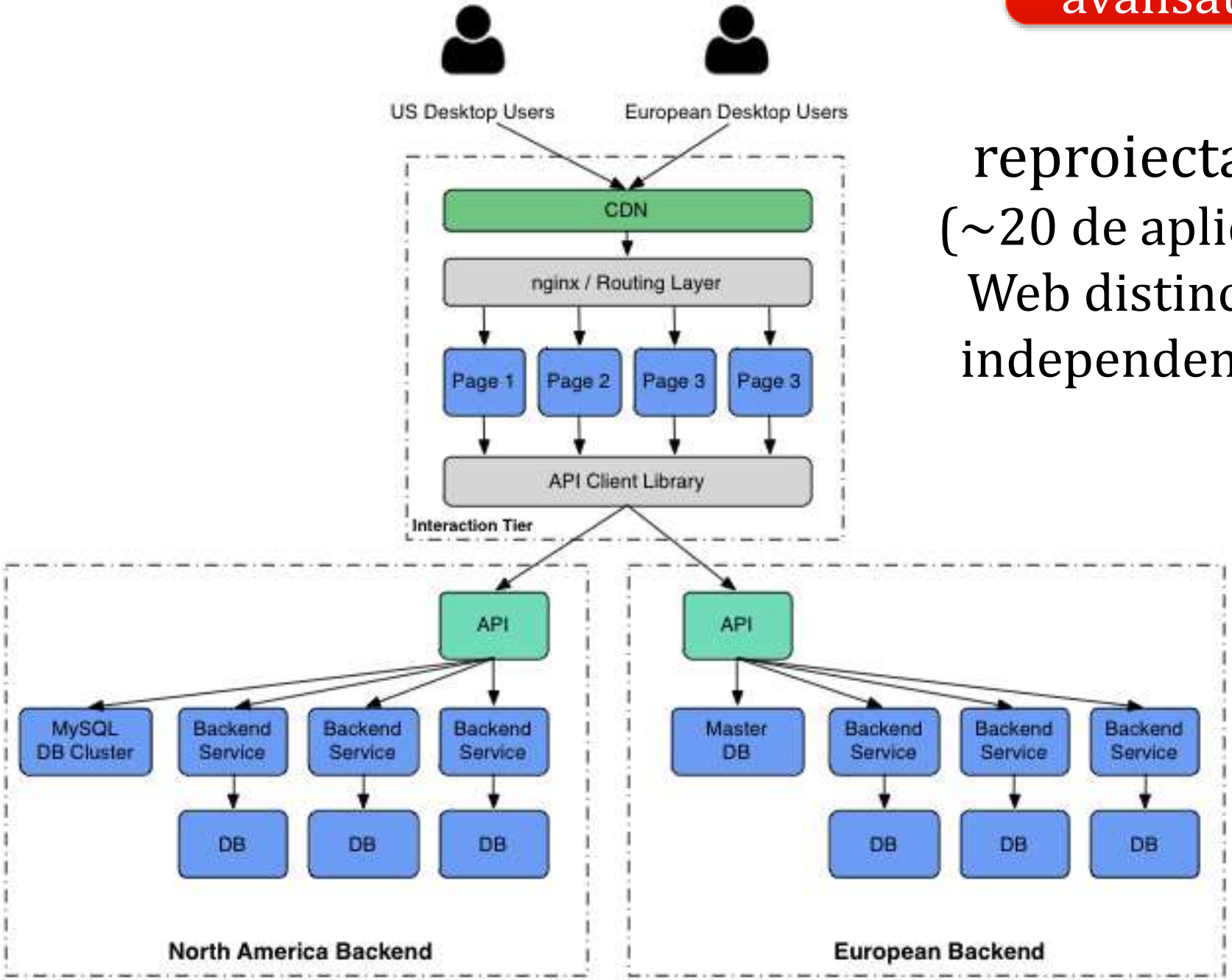
implementări distincte în funcție de zona geografică

avansat



oferirea suportului
pentru interacțiuni cu dispozitive mobile
API dedicat, accesul depinzând de localizarea utilizatorului

avansat



reproiectare (~20 de aplicații Web distincte, independente)

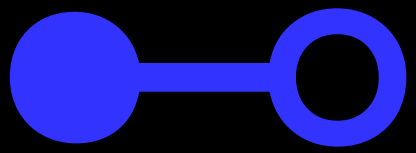
avansat

soap *vs.* rest

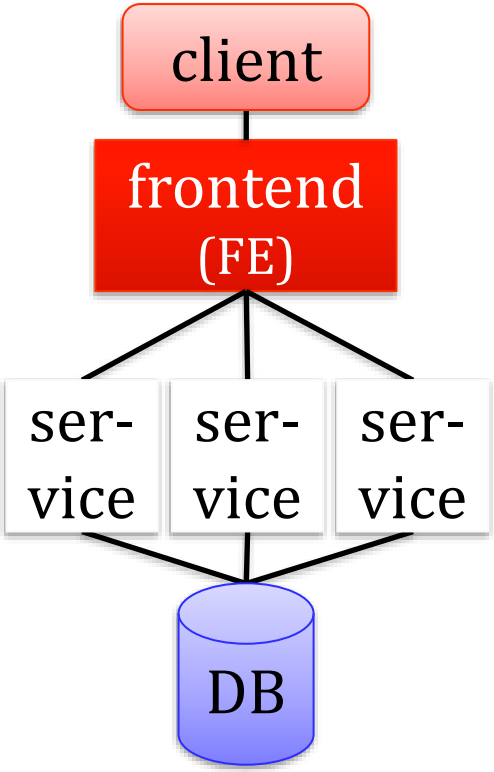
SOAP	REST
Acțiuni arbitrare (verbe)	Acțiuni fixe – HTTP: GET, POST,…
Structuri de date oricât de complexe – inclusiv validare	Operează asupra reprezentărilor de resurse – XML, JSON, HTML
Descriere complexă a serviciului (pe baza WSDL)	Scalabil (mai ușor de extins) Descriere via OpenAPI Spec.
Suport pentru *XML messaging*	Bazat pe URI + hipermedia
Dezvoltare sofisticată: securitate, intermediari, specificații WS-*, interoperabilitate,…	Uzual, mai facil de programat (+disponibilitatea API-urilor)
Specific mediului *enterprise* (infrastructuri complexe)	Abordare pragmatică aplicații sociale *et al.* (Web 2.0)

rezumat

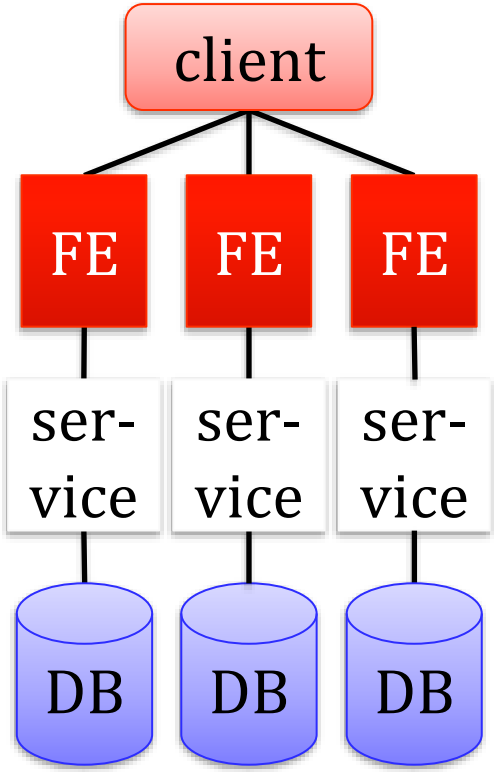
dezvoltare de servicii Web via REST



de la HTTP la metodologii, instrumente și exemple



arhitectură bazată
pe servicii Web



arhitectură recurgând
la micro-servicii

episodul viitor:

dezvoltarea de aplicații Web complexe: specificarea API-urilor, micro-servicii, autentificare, autorizare, acces la date via GraphQL